華中科技大學

《离散数学》编程实践

实 验 报 告

成　　绩：
评　　语：
教师签字：
评阅日期：

专业班级：计算机类 2510 班

学　　号：U202515003

姓　　名：王皓轩

指导教师：戴小海、金燕

完成日期：2026 年 6 月 8 日

计算机科学与技术学院

2026 年 6 月

目 录

1 数理逻辑 1

1.1 实验目的 1
1.2 实验内容 1
1.2.1 推理与证明 1
1.2.2 命题逻辑 2
1.2.3 逻辑与 AI 表示 2
1.3 实验过程 3
1.3.1 推理与证明 3
1.3.2 命题逻辑 7
1.3.3 逻辑与 AI 表示 9

2 集合及其运算 11

2.1 实验目的 11
2.2 实验内容 11
2.2.1 集合的表示和基本运算 11
2.3 实验过程 11
2.3.1 集合的表示和基本运算 11

3 关系表示及其运算 14

3.1 实验目的 14
3.2 实验内容 14
3.2.1 二元关系 14
3.2.2 特殊关系 14
3.3 实验过程 15
3.3.1 二元关系 15
3.3.2 特殊关系 19

4 函数 22

4.1 实验目的 22
4.2 实验内容 22
4.2.1 函数的判断 22
4.2.2 函数的运算 22
4.3 实验过程 23
4.3.1 函数的判断 23
4.3.2 函数的运算 24

5 初等数论 27

5.1 实验目的 27
5.2 实验内容 27
5.2.1 除法和模运算 27

5.2.2 素数和最大公因数 27
5.2.3 同余方程 28
5.2.4 密码学 28
5.3 实验过程 28
5.3.1 除法和模运算 28
5.3.2 素数和最大公因数 30
5.3.3 同余方程 33
5.3.4 密码学 35
6 组合数学 38
6.1 实验目的 38
6.2 实验内容 38
6.2.1 计数基础 38
6.2.2 鸽巢原理 38
6.2.3 排列与组合 39
6.2.4 生成函数 39
6.3 实验过程 40
6.3.1 计数基础 40
6.3.2 鸽巢原理 42
6.3.3 排列与组合 45
6.3.4 生成函数 46
7 图论 49
7.1 实验目的 49
7.2 实验内容 49
7.2.1 树 49
7.2.2 图论的相关定义 49
7.2.3 图论的应用 49
7.2.4 特殊图 50
7.3 实验过程 50
7.3.1 树 50
7.3.2 图论的相关定义 51
7.3.3 图论的应用 54
7.3.4 特殊图 57
8 数论（选做） 60
8.1 实验目的 60
8.2 实验内容 60
8.2.1 数论 60
8.3 实验过程 61
8.3.1 数论 61
9 总结与课程建议 67
9.1 课程总结 67

9.2 问题及建议 68
9.2.1 存在的问题 68
9.2.2 改进建议 68

1　数理逻辑

1.1 实验目的

（1）掌握命题逻辑的基本符号化方法，能够将自然语言描述的逻辑规则、推理形式转化为标准命题公式。

（2）学会使用 Python 的 sympy 库实现命题逻辑运算、等价演算、推理有效性验证，理解重言式、矛盾式、可满足式的判定方法。

（3）验证经典推理形式（构造性二难、归结原理）的有效性，识别逻辑谬误（肯定后件）并构造反例，加深对正确推理规则的理解。

（4）实现命题逻辑真值表自动生成与逻辑等价律（分配律）的验证，掌握计算机自动推理的基础思想。

（5）掌握谓词逻辑中全称量词、存在量词的使用，能够将 AI 知识库中的规则与事实符号化，建立逻辑与人工智能表示的关联。

1.2 实验内容

1.2.1 推理与证明

（1）构造二难推理的有效性验证。在逻辑学中，存在许多被证明是永远有效的推理形式，它们可以作为我们进行正确推理的“模板”。构造性二难（Constructive Dilemma）就是其中一种常见的有效推理形式。本关任务是使用 sympy 库编程来验证构造性二难推理的有效性。其结构如下：

前提 1: P >> Q (如果 P，那么 Q)

前提 2: R >> S (如果 R，那么 S)

前提 3: P | R (P 或 R)

结论: Q | S (因此，Q 或 S)

（2）识别无效推理并寻找反例。在逻辑推理中，除了有效的推理形式，还存在许多看似正确但实际上是错误的推理形式,这些被称为逻辑谬误(Fallacy)。一个常见的逻辑谬误是“肯定后件”(Affirming the consequent)。

本关任务是：

1.首先证明“肯定后件”这个推理形式是无效的。

2.找出一个反例（即一组能使所有前提为真，但结论为假的赋值）来证明其无效性。

推理结构如下：

前提 1: P >> Q (如果 P，那么 Q)

前提 2: Q (Q 成立)

结论: P (因此，P 成立)

（3）归结原理的应用。归结（Resolution）是自动定理证明（Automated Theorem Proving）领域中的一个核心推理规则，尤其在基于逻辑的 AI 系统中被广泛使用。它允许计算机通过一个简单的规则来系统地推导出新的结论。本关任务是验证归结原理的基本形式是有效的。其结构如下：

前提 1: P | Q

前提 2: ~P | R

结论: Q | R

（4）用等价演算进行证明。本关任务是证明 ~(P >> Q) 与 P & ~Q 这两个命题公式是逻辑等价的。我们将不使用真值表，而是使用 sympy 的符号计算能力来模拟等价演算的过程，即通过一系列已知的等价规则（如德·摩根定律）进行逐步化简。

1.2.2 命题逻辑

（1）三变量真值表生成器。编写一个程序，输入一个包含 P、Q、R 三个命题变量的逻辑表达式字符串，程序能自动生成并打印该表达式的完整真值表。

注：数理逻辑部分的练习题需要使用一些简单的 python 语法，建议通过询问常见的大语言模型快速上手。

（2）分配律的验证。本关任务：逻辑等价中的分配律是化简表达式的重要工具。请使用 sympy 库编写一个程序，验证以下两个分配律是否为重言式（永真式），从而证明它们是成立的。

1. P & (Q | R) <=> (P & Q) | (P & R)

2. P | (Q & R) <=> (P | Q) & (P | R)

1.2.3 逻辑与 AI 表示

（1）全称规则。假设你正在为一个人工智能构建知识库，它需要理解关于

世界的一些普遍规则。本关任务是将以下两条自然语言规则，符号化为谓词逻辑表达式的字符串，以便存入机器人的知识库。

1.“所有的矩形都是四边形。”

2.“没有不遵守交通规则的司机。”

（2）存在事实。除了需要理解普遍规则外，一个 AI 知识库也需要存储关于世界的具体事实。本关任务是将以下两条自然语言描述的事实，符号化为谓词逻辑表达式的字符串，以便 AI 能够理解和使用。

1.“有些机器人会编程。”

2.“并非所有金属都能导电。”（等价于“有些金属不能导电”）

1.3 实验过程

1.3.1 推理与证明

（1）构造二难推理的有效性验证

解题思路：按照构造性二难的逻辑形式编写前提与结论代码，拼接成 “前提合取→结论” 的整体表达式，利用重言式判定规则：原式永真等价于其否定无成真赋值，调用 sympy 的 DPLL 算法检验否定式可满足性，若无可行赋值即可确定推理有效。

算法复杂度分析：

1.时间复杂度：核心采用 DPLL 可满足性判定算法，最坏情况下时间复杂度为 O (2^n)，其中 n 为命题变元个数。本问题中 n=4，仅需检查 16 种可能的真值组合，实际运行时间可忽略不计。

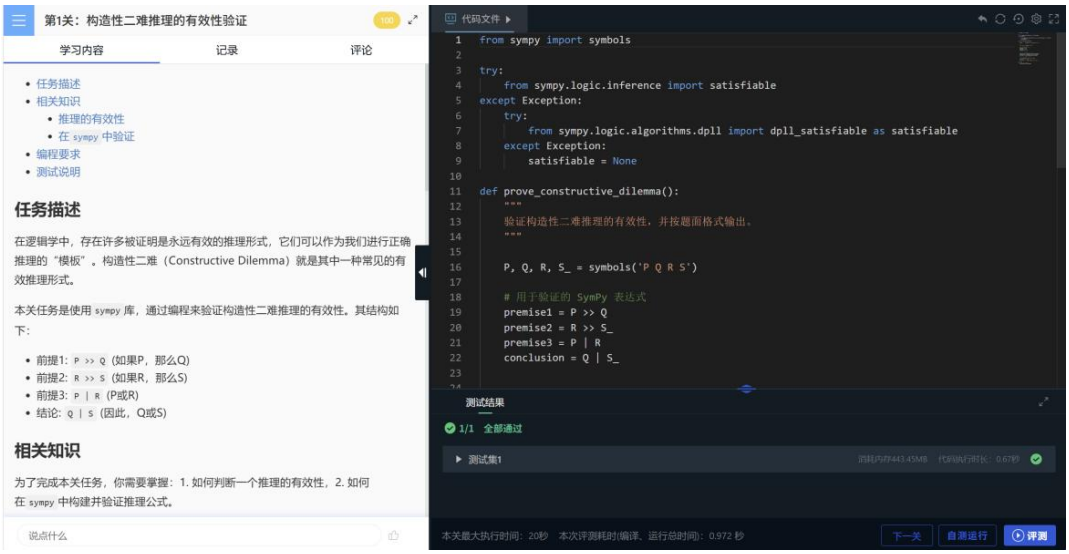
2.空间复杂度：O (n)，主要用于存储命题变元、逻辑表达式以及 DPLL 算法递归调用栈，空间开销与变元个数线性相关。

核心代码片段：

```
# 定义前提与结论
premise1 = P >> Q
premise2 = R >> S_
premise3 = P | R
conclusion = Q | S_

# 构造推理蕴含式并验证有效性
inference = (premise1 & premise2 & premise3) >> conclusion
is_valid = not satisfiable(~inference)  # 若否定不可满足，则原推理有效
```

截图证明：



（2）识别无效推理并寻找反例

解题思路：先写出肯定后件逻辑式(P→Q)∧Q→P，用代码定义两个前提与结论，拼接整体蕴含式；通过判定式子的否定是否可满足判断推理有效性，再构造前提真、结论假的表达式查找赋值反例，以此证明推理无效。

算法复杂度分析：

1.时间复杂度：底层依靠 DPLL 算法，最坏时间复杂度 $O(2^n)$，本问题仅包含 P、Q 两个命题变元，总共只有 4 组真值组合，运算规模极小。

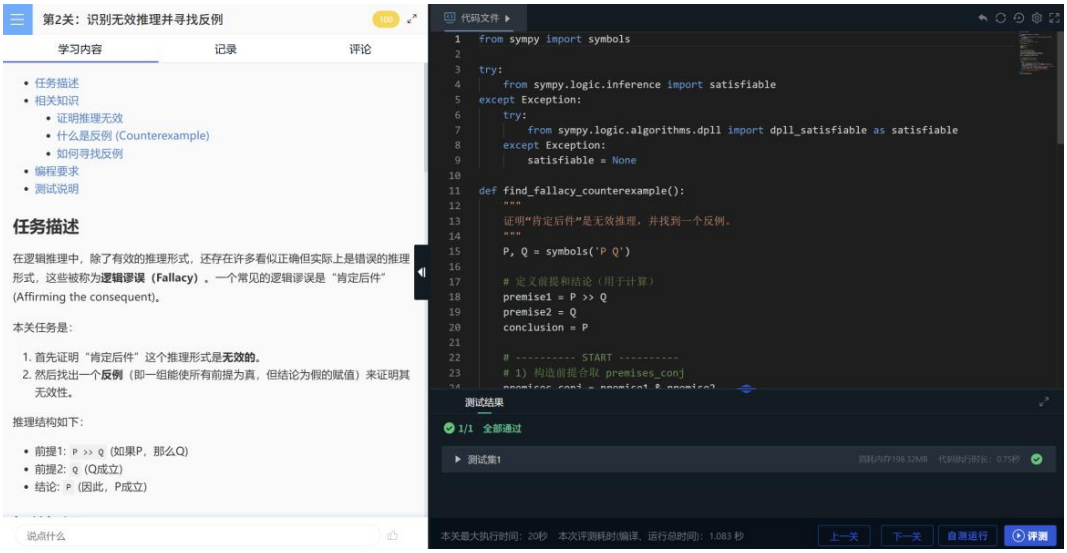
2.空间复杂度：O(n)，空间开销集中在存储命题变量与逻辑表达式，占用空间随变元数量线性变化。

核心代码片段：

```
premise1 = P >> Q
premise2 = Q
conclusion = P

inference = (premise1 & premise2) >> conclusion
is_valid = not satisfiable(~inference)
#  求解前提全真、结论为假的反例赋值
counter = satisfiable(premise1 & premise2 & ~conclusion)
```

截图证明：



（3）归结原理的应用

解题思路：代码先定义归结原理的两个前提和结论，将前提合取后与结论的否定进行组合，通过检查该组合式是否可满足验证推理有效性。若该组合式不可满足，说明前提为真时结论一定为真，归结原理有效。

算法复杂度分析：

1. 时间复杂度：底层 DPLL 算法最坏复杂度 $O(2^n)$，本题共 3 个命题变量，仅有 8 组真值组合，计算量很小。

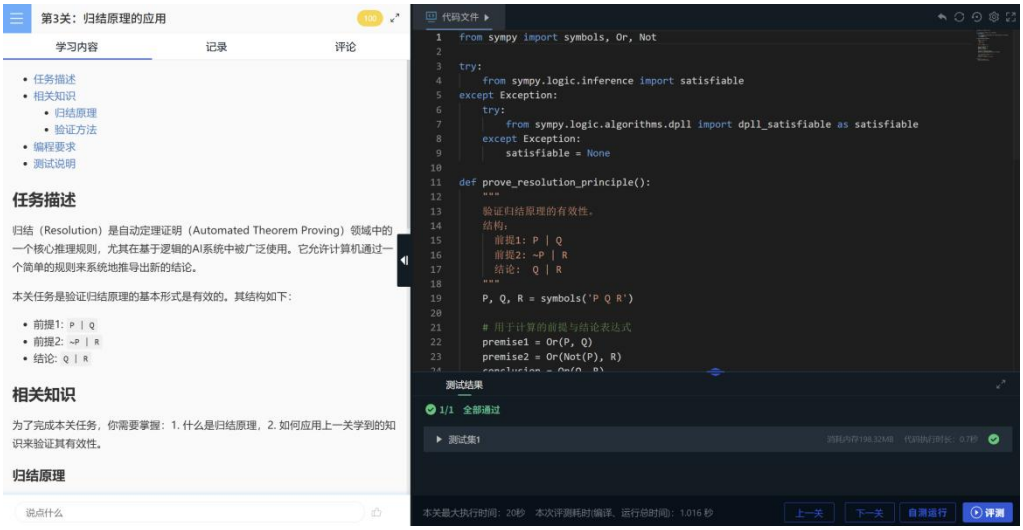
2. 空间复杂度：O(n)，仅保存命题变量与逻辑算式，空间占用随变量数量线性变化。

核心代码片段：

```
# 定义归结原理的两个前提
premise1 = Or(P, Q)
premise2 = Or(Not(P), R)
# 定义归结原理的结论
conclusion = Or(Q, R)

# 前提合取
premises_conj = premise1 & premise2
# 验证有效性:前提为真且结论为假不可满足，则推理有效
is_valid = not satisfiable(premises_conj & ~conclusion)
```

截图证明：



（4）用等价演算进行证明

解题思路：代码定义两个待验证等价的逻辑表达式，通过 sympy 的逻辑化简函数对第一个表达式进行化简，直接对比化简结果与第二个表达式是否完全一致，结果相等即可证明两个原式逻辑等价。

算法复杂度分析：

1. 时间复杂度：逻辑化简基于布尔代数规则，复杂度为常数级 O(1)，运算极快。

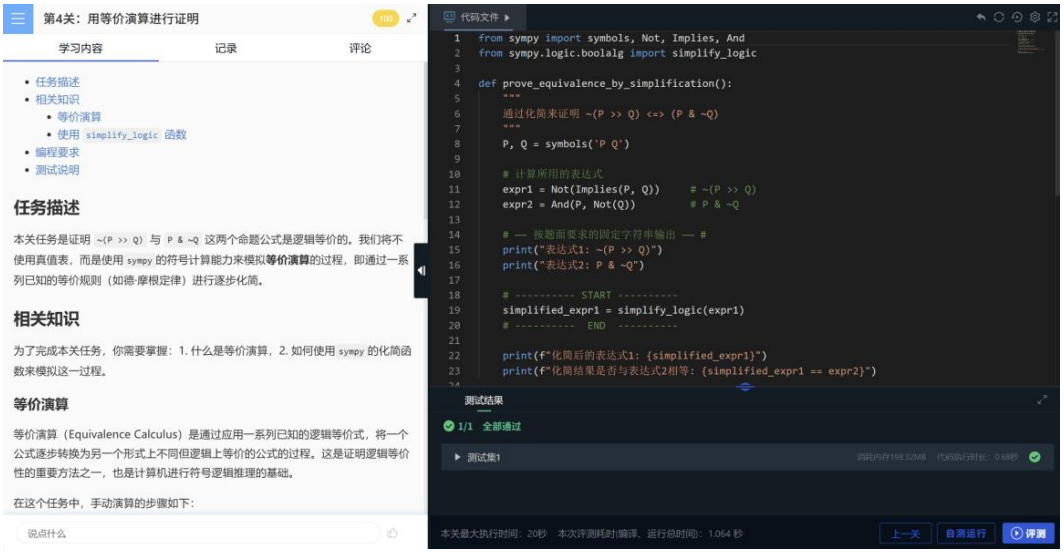
2. 空间复杂度：O(1)，仅存储两个逻辑表达式与化简结果，占用空间固定。

核心代码片段：

```
# 定义两个待证明等价的逻辑表达式
expr1 = Not(Implies(P, Q))
expr2 = And(P, Not(Q))

# 对第一个表达式进行逻辑化简
simplified_expr1 = simplify_logic(expr1)
# 比较化简结果与第二个表达式是否相等
is_equivalent = (simplified_expr1 == expr2)
```

截图证明：



1.3.2 命题逻辑

（1）三变量真值表生成器

解题思路：本任务实现 P、Q、R 三个命题变元的真值表生成，先定义变量并解析输入的逻辑表达式，遍历所有 8 种真假组合，依次代入表达式计算结果，最后按格式输出每一组变量取值与对应结果，完成真值表打印。

算法复杂度分析：

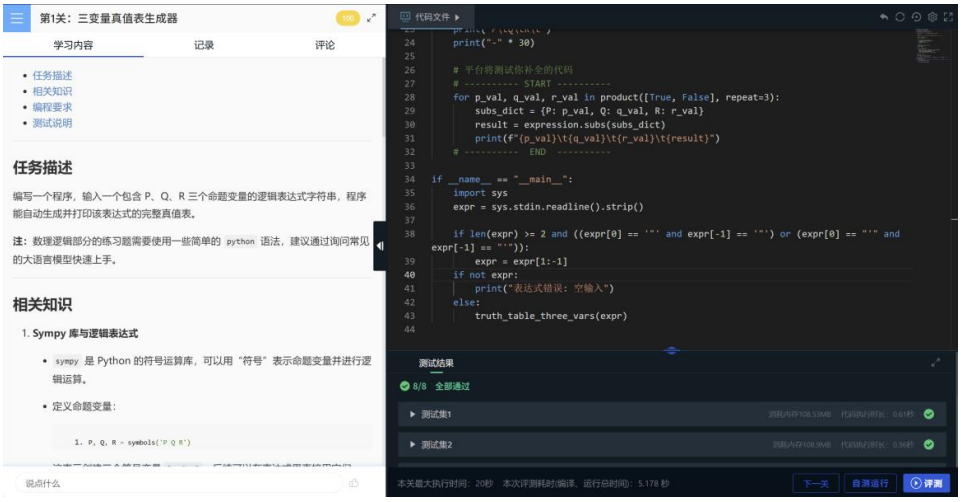
1. 时间复杂度：O（1），固定 3 个变量，仅遍历 8 次组合，运算量固定

2. 空间复杂度：O（1），仅存储表达式和变量赋值，占用空间固定

核心代码片段：

```
# 遍历 P、Q、R 所有 8 种真假组合
for p_val, q_val, r_val in product([True, False], repeat=3):
    # 构造变量赋值字典
    subs_dict = {P: p_val, Q: q_val, R: r_val}
    # 代入表达式计算结果
    result = expression.subs(subs_dict)
    # 打印当前赋值与计算结果
    print(f"{p_val}\t{q_val}\t{r_val}\t{result}")
```

截图证明：



（2）分配律的验证

解题思路：本任务验证两条逻辑分配律，分别构造 P & (Q | R) 与 (P & Q) | (P & R)、P | (Q & R) 与 (P | Q) & (P | R) 的等价式，通过判断等价式的否定是否不可满足，验证两条分配律都是永真的重言式。

算法复杂度分析：

1. 时间复杂度：O (1)，固定 3 个变量，仅做简单逻辑运算与可满足性判定

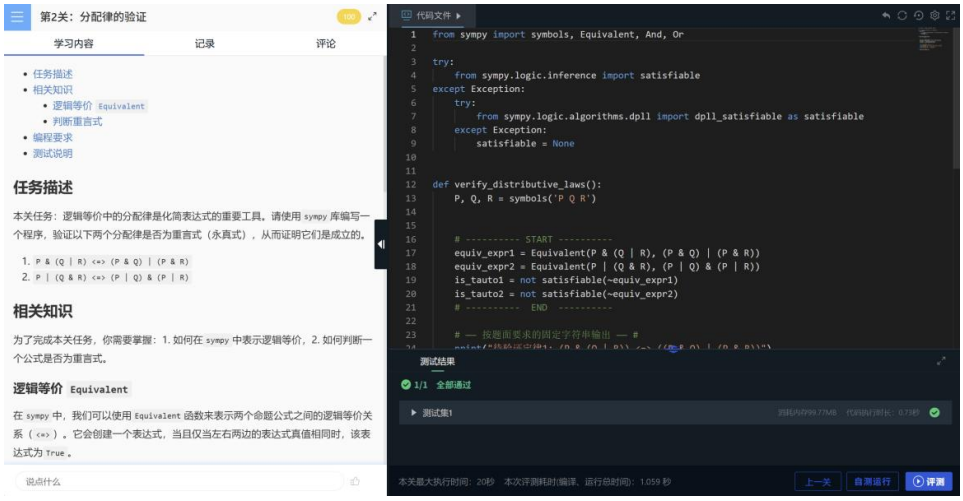
2. 空间复杂度：O (1)，仅存储逻辑表达式与判定结果，占用空间固定

核心代码片段：

```
# 构造两条分配律的等价表达式
equiv_expr1 = Equivalent(P & (Q | R), (P & Q) | (P & R))
equiv_expr2 = Equivalent(P | (Q & R), (P | Q) & (P | R))

# 判定是否为重言式：否定不可满足则为真
is_tauto1 = not satisfiable(~equiv_expr1)
is_tauto2 = not satisfiable(~equiv_expr2)
```

截图证明：



1.3.3 逻辑与 AI 表示

（1）全称规则

解题思路：将自然语言描述的全称规则转化为谓词逻辑表达式，所有矩形都是四边形对应全称量词下的蕴含式，没有不遵守交通规则的司机等价于所有司机都遵守交通规则，同样采用全称量词加蕴含式完成符号化。

算法复杂度分析：

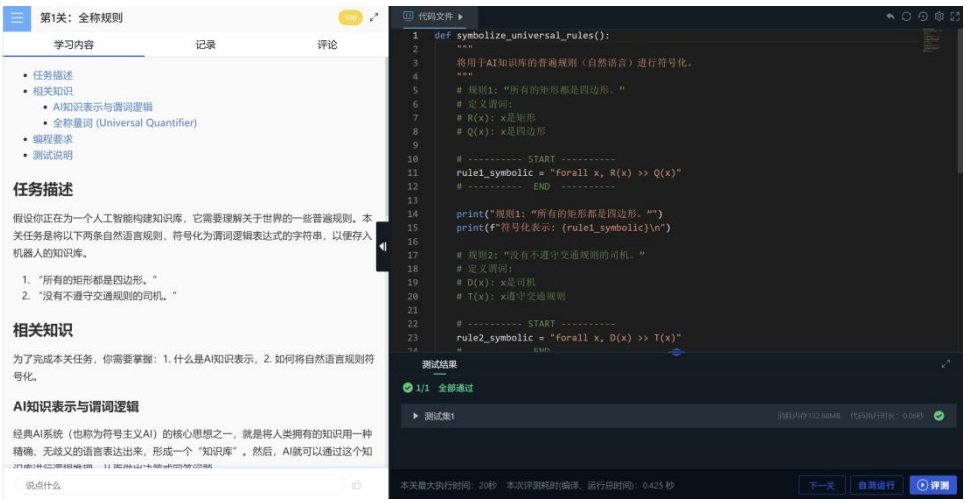
1. 时间复杂度：0（1），仅进行字符串拼接与赋值，无循环运算

2. 空间复杂度：0（1），仅存储两个符号化结果，占用空间固定

核心代码片段：

```
# 所有矩形都是四边形：任意 x，如果 x 是矩形，那么 x 是四边形
rule1_symbolic = "forall x, R(x) >> Q(x)"
# 所有司机都遵守交通规则：任意 x，如果 x 是司机，那么 x 遵守交通规则
rule2_symbolic = "forall x, D(x) >> T(x)"
```

截图证明：



（2）存在事实

解题思路：对存在性事实进行符号化，有些机器人会编程使用存在量词加合取式表示，并非所有金属都能导电转化为存在金属不能导电，同样使用存在量词与合取式完成符号化。

算法复杂度分析：

1. 时间复杂度：0（1），仅进行字符串赋值，运算量固定

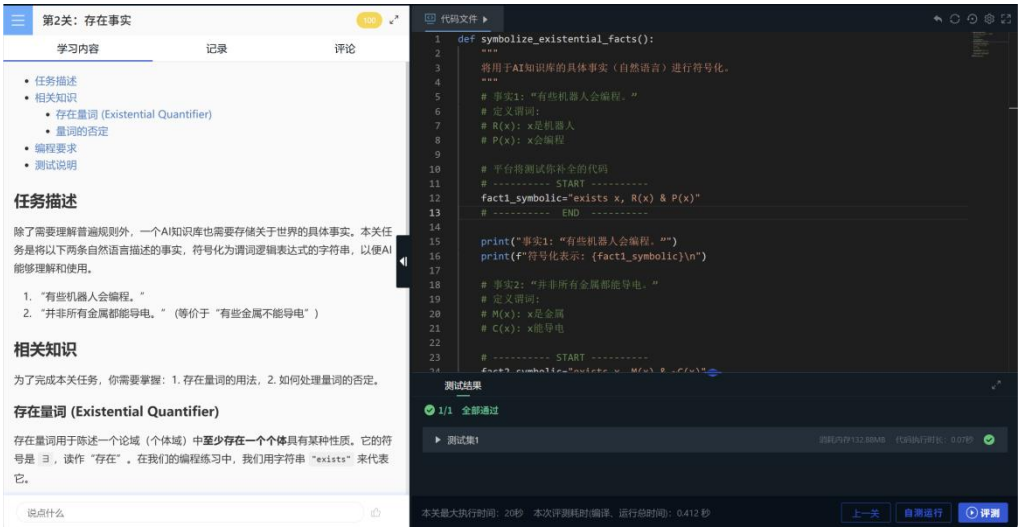
2. 空间复杂度：0（1），仅存储两个符号化结果，占用空间固定

核心代码片段：

```
# 有些机器人会编程：存在 x，x 是机器人且 x 会编程
```

```
fact1_symbolic = "exists x, R(x) & P(x)"
# 并非所有金属都能导电：存在 x，x 是金属且 x 不能导电
fact2_symbolic = "exists x, M(x) & ~C(x)"
```

截图证明：



2　集合及其运算

2.1 实验目的

（1）理解集合、子集、幂集的核心定义，掌握集合的计算机表示方法与幂集的生成算法。

（2）熟练应用容斥原理解决有限集合的计数问题，理解多重集合交集的计数规则。

（3）掌握集合基本运算的编程实现方法，提升数学集合概念向程序逻辑转化的能力。

（4）分析集合运算算法的时间与空间复杂度，建立算法效率意识。

2.2 实验内容

2.2.1 集合的表示和基本运算

（1）求集合的幂集。本关任务：编写一个程序，求出一个集合的幂集

（2）容斥原理。本关任务：给定一个数 n 和 m 个质数，要求利用容斥原理求出 1~n 中有多少数至少被这 m 个数中的至少一个整除。

2.3 实验过程

2.3.1 集合的表示和基本运算

（1）求集合的幂集

解题思路：先输出空集，再按子集元素个数从 1 到 n 依次生成所有非空子集。用下标数组保存当前组合，按字典序输出后，不断寻找下一个组合：找到可递增的位置 j，让 p [j] 加 1，后面的位置依次设为 p [m]=p [m-1]+1，直到所有组合生成完毕，最终得到完整幂集。

算法复杂度分析：

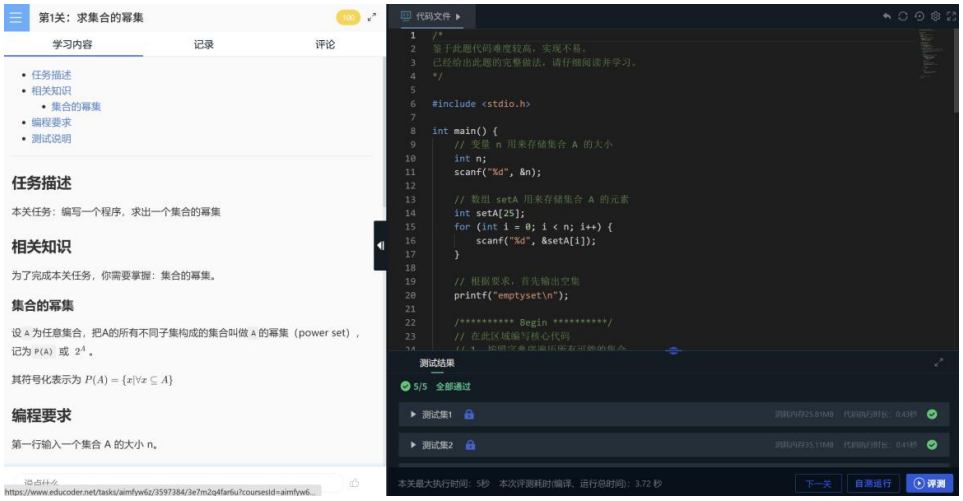
1.时间复杂度：O (n*2^n)，需要生成 2^n 个子集，每个子集平均处理 n 个元素

2.空间复杂度：O (n)，只使用固定大小的数组存储元素和组合下标

核心代码片段：

```
// 按元素个数 k 枚举所有子集
for (int k = 1; k <= n; k++) {
    int p[25];
    // 初始化第一个 k 元组合
    for (int i = 0; i < k; i++)
        p[i] = i;
    while (1) {
        // 打印当前组合
        printf("{");
        for (int i = 0; i < k; i++) {
            printf("%d", setA[p[i]]);
            if(i < k - 1) printf(",");
        }
        printf("}\n");
        // 查找可以更新的下标 j
        int j = k - 1;
        while (j >= 0 && p[j] == n - k + j)
            j--;
        if (j < 0) break;
        p[j]++;
        // 后面下标依次设为前一个加 1
        for (int m = j + 1; m < k; m++)
            p[m] = p[m - 1] + 1;
    }
}
```

截图证明：



（2）容斥原理

解题思路：使用二进制掩码枚举所有质数的非空子集，统计每个子集的质数个数与乘积，根据个数奇偶性做加减运算：奇数个质数相加，偶数个质数相减，

最终得到 1 到 n 中能被任意质数整除的数字总数。

算法复杂度分析：

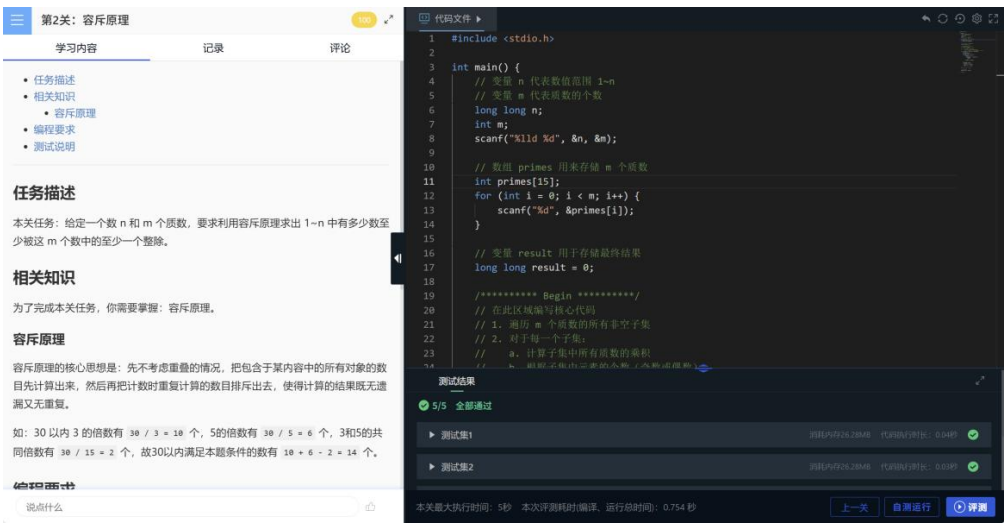
1. 时间复杂度：O（m*2^m），枚举 2^m 个子集，每个子集遍历 m 个质数

2. 空间复杂度：O（m），仅存储质数数组，使用常数临时变量

核心代码片段：

```
// 枚举所有非空子集
for (int mask = 1; mask < (1 << m); mask++) {
        int cnt = 0;
        long long product = 1;
        int valid = 1;
        for (int i = 0; i < m; i++) {
                if (mask & (1 << i)) {
                        cnt++;
                        // 防止乘积溢出
                        if (product > n / primes[i]) {
                                valid = 0;
                                break;
                        }
                        product *= primes[i];
                }
        }
        if (!valid) continue;
        // 容斥原理:奇加偶减
        if (cnt % 2 == 1)
                result += n / product;
        else
                result -= n / product;
}
```

截图证明：



3　关系表示及其运算

3.1 实验目的

（1）理解二元关系、笛卡尔积、关系矩阵的定义，掌握关系的矩阵表示方法。

（2）实现关系的基本运算：幂运算、传递性判定、传递闭包计算（Warshall 算法）。

（3）掌握等价关系、同余关系、偏序关系的性质，能够求解等价类、商集，识别偏序集中的特殊元素（最大元、最小元、极大元、极小元）。

（4）理解关系运算在离散结构中的应用，掌握关系矩阵运算的编程实现。

3.2 实验内容

3.2.1 二元关系

（1）笛卡尔积。本关任务：编写一个程序，实现集合笛卡尔积运算，并按字典序输出。

（2）关系矩阵。给定从集合 A 到集合 B 的二元关系 R，输出其关系矩阵。

（3）关系的幂运算。给定关系 R 的关系矩阵，求其 m 次幂 R^m。

（4）关系的传递性。本关任务：给定一个关系矩阵，判断其是否具有传递性。

（5）关系的闭包运算。本关任务：给定一个关系矩阵，输出其传递闭包。建议使用 Warshall 算法 解决此问题。

3.2.2 特殊关系

（1）等价类与商集。给定集合 A={0,1,2,…,n} 和整数 m(m≤n)，R 是 A 上以 m 为模的同余关系。求集合 A 关于 R 的商集，即 A/R。

（2）偏序关系与特殊元素。给定集合 A={1,2,…,n} 上的偏序关系 R，再给定 A 的子集 B，求出 B 的最大元、最小元、极大元、极小元。

3.3 实验过程

3.3.1 二元关系

（1）笛卡尔积

解题思路：先读取集合 A 和集合 B 的元素，通过双层循环遍历 A 中每一个元素与 B 中每一个元素配对，按照字典序依次输出所有序偶，完成集合笛卡尔积的运算与打印。

算法复杂度分析：

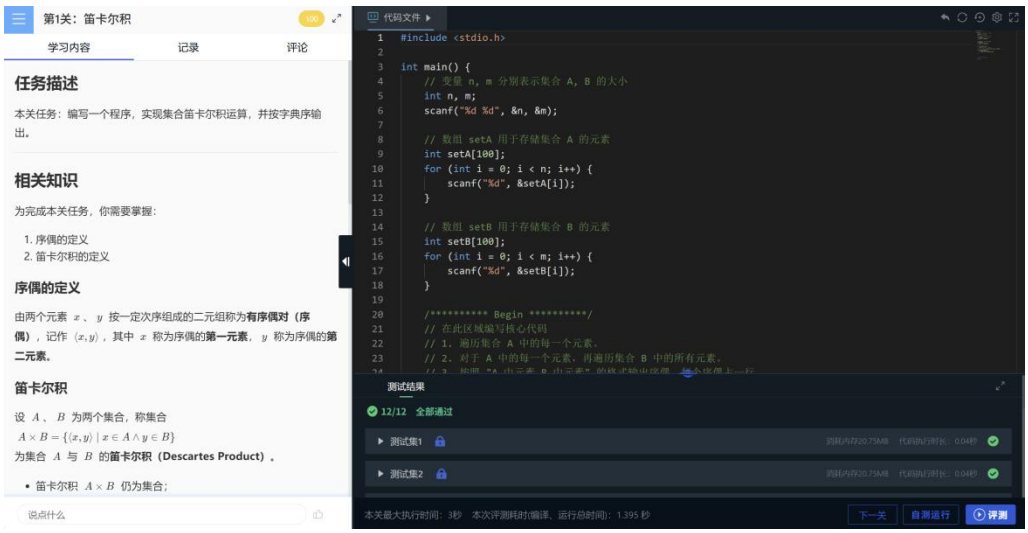
1.时间复杂度：O（n*m），两层循环分别遍历 n 个和 m 个元素

2.空间复杂度：O（n+m），仅存储两个集合的元素

核心代码片段：

```
// 双层循环生成所有笛卡尔积序偶
for(int q=0;q<=n-1;q++){
        for(int w=0;w<=m-1;w++){
                printf("%d %d\n",setA[q],setB[w]);
        }
}
```

截图证明：



（2）关系矩阵

解题思路：初始化全 0 矩阵，读取 k 个关系序偶，将输入的 1 起始序号转换为 0 起始数组下标，把对应矩阵位置置为 1，最终输出完整关系矩阵。

算法复杂度分析：

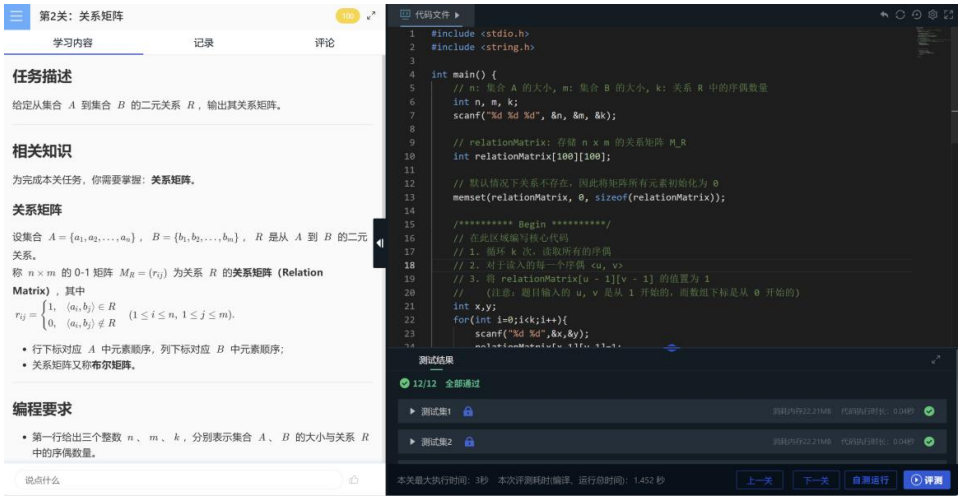
1.时间复杂度：O（k），循环读取 k 个序偶并赋值

2.空间复杂度：O（n*m），存储 n 行 m 列的关系矩阵

核心代码片段：

```
// 读取序偶并设置对应矩阵位置为 1
int x,y;
for(int i=0;i<k;i++){
        scanf("%d %d",&x,&y);
        relationMatrix[x-1][y-1]=1;
}
```

截图证明：



（3）关系的幂运算

解题思路：处理幂次为 0 时输出单位矩阵，幂次为 1 时直接复制原矩阵，幂次大于 1 时通过布尔矩阵乘法迭代计算，逐位判断路径是否存在，将结果存入矩阵得到 R 的 m 次幂。

算法复杂度分析：

1. 时间复杂度：O（$m*n^3$），迭代 m-1 次，每次三层 n 阶循环

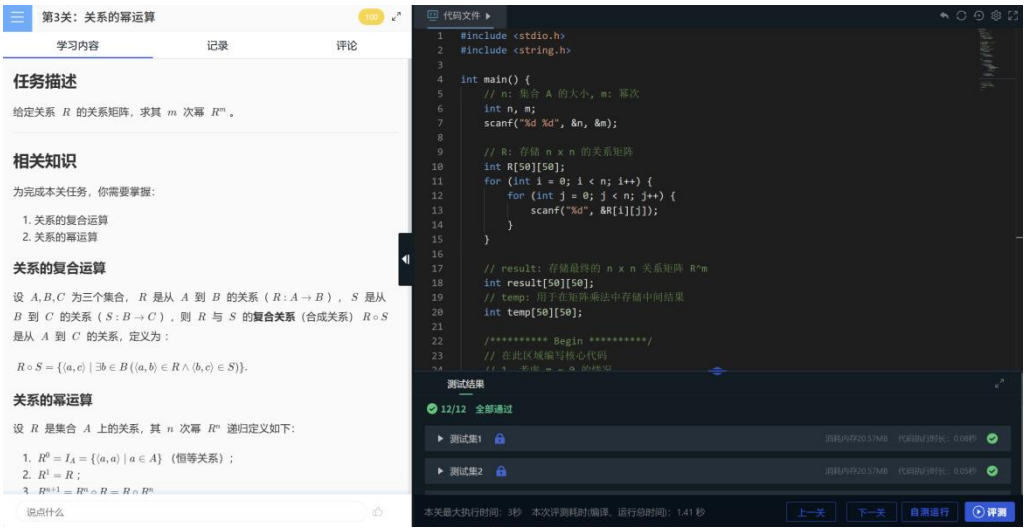
2. 空间复杂度：O（n^2），存储原矩阵、结果矩阵和临时矩阵

核心代码片段：

```
// 布尔矩阵乘法迭代计算关系幂
for(int i=0;i<m-1;i++){
        for(int j=0;j<n;j++){
                for(int k=0;k<n;k++){
                        for(int q=0;q<n;q++){
                                if(temp[j][q]*R[q][k]!=0){
                                        result[j][k]=1;
                                        break;
                                }
                                result[j][k]=0;
                        }
```

```
            }
        }
        // 更新临时矩阵为当前结果
        for(int u=0;u<n;u++){
            for(int h=0;h<n;h++){
                temp[u][h]=result[u][h];
            }
        }
    }
```

截图证明：



（4）关系的传递性

解题思路：使用三层循环遍历所有 i,j,k 组合，检查是否满足 matrix[i][j]=1 且 matrix[j][k]=1 但 matrix[i][k]=0，若存在则关系不传递，直接跳出循环输出结果。

算法复杂度分析：

1. 时间复杂度：O（n^3），三层循环遍历全部下标组合

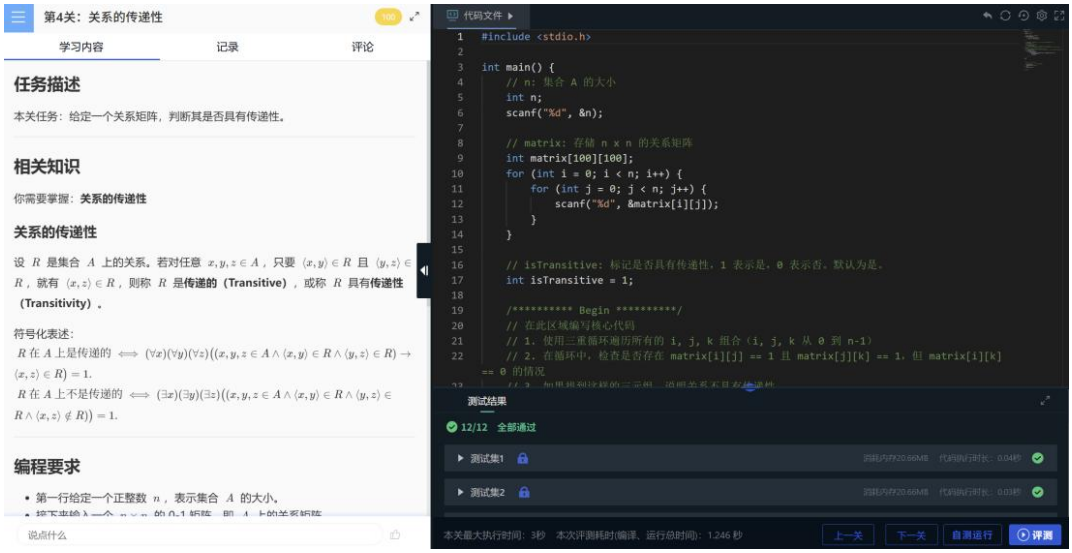
2. 空间复杂度：O（n^2），仅存储关系矩阵

核心代码片段：

```
// 遍历检查传递性条件
for(int i=0;i<n;i++){
    for(int j=0;j<n;j++){
        for(int k=0;k<n;k++){
            if(matrix[i][j] == 1&&matrix[j][k] == 1&&matrix[i][k] == 0){
                isTransitive=0;
                goto end;
            }
        }
```

```
        }
    }
    end:
```

截图证明：



（5）关系的闭包运算

解题思路：采用 Warshall 算法，以 k 为中间节点，遍历所有 i，j，若 i 到 k、k 到 j 都可达，则将 i 到 j 设为可达，逐步更新矩阵得到传递闭包。

算法复杂度分析：

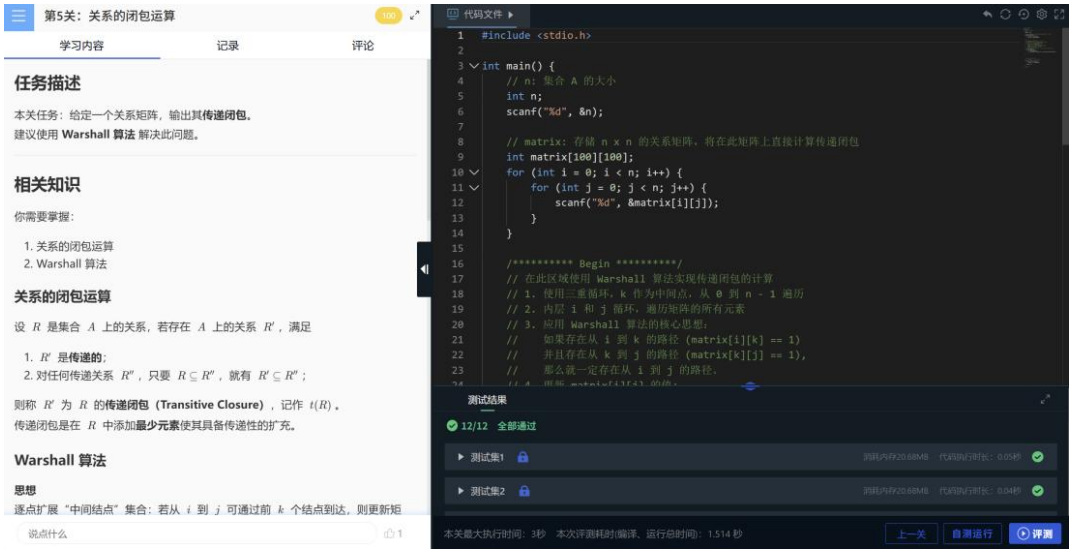
1. 时间复杂度：O（n^3），标准 Warshall 算法三层循环

2. 空间复杂度：O（n^2），直接在原矩阵上更新

核心代码片段：

```
// Warshall 算法求传递闭包
for(int k=0;k<n;k++){
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            matrix[i][j] = matrix[i][j] || (matrix[i][k] && matrix[k][j]);
        }
    }
}
```

截图证明：



3.3.2 特殊关系

（1）等价类与商集

解题思路：按模 m 的余数划分等价类，外层循环遍历余数 0 到 m-1，内层按步长 m 遍历集合元素，输出同余的所有元素，每行代表一个等价类。

算法复杂度分析：

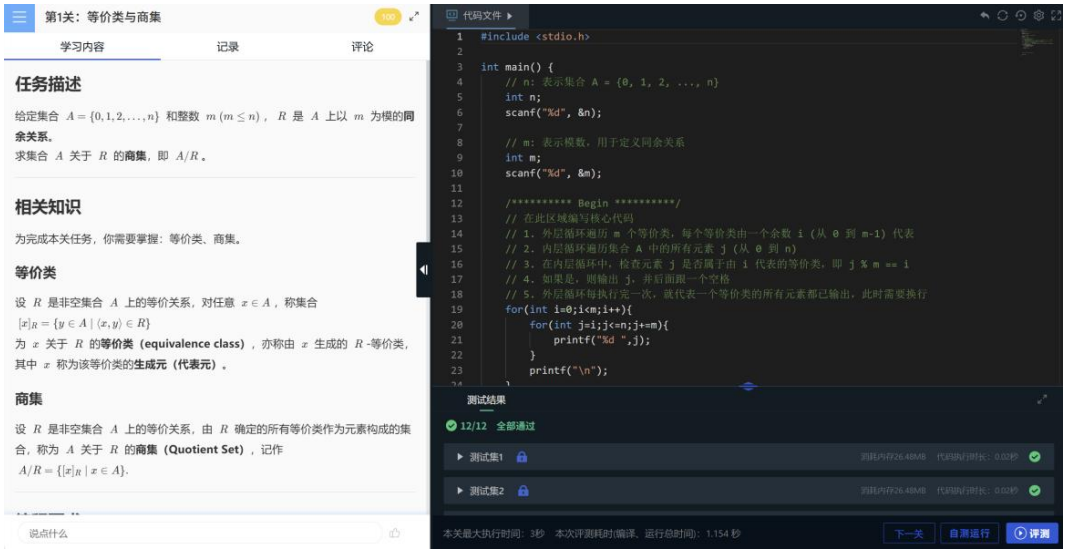
1. 时间复杂度：O（n），遍历 0 到 n 所有元素一次

2. 空间复杂度：O（1），无额外数组存储

核心代码片段：

```
// 按模 m 输出所有等价类
for(int i=0;i<m;i++){
        for(int j=i;j<=n;j+=m){
                printf("%d ",j);
        }
        printf("\n");
}
```

截图证明：



（2）偏序关系与特殊元素

解题思路：先遍历子集元素找出极大元、极小元，再判断唯一的极大元 / 极小元是否满足与所有元素可比，确定最大元、最小元，最后排序输出结果。

算法复杂度分析：

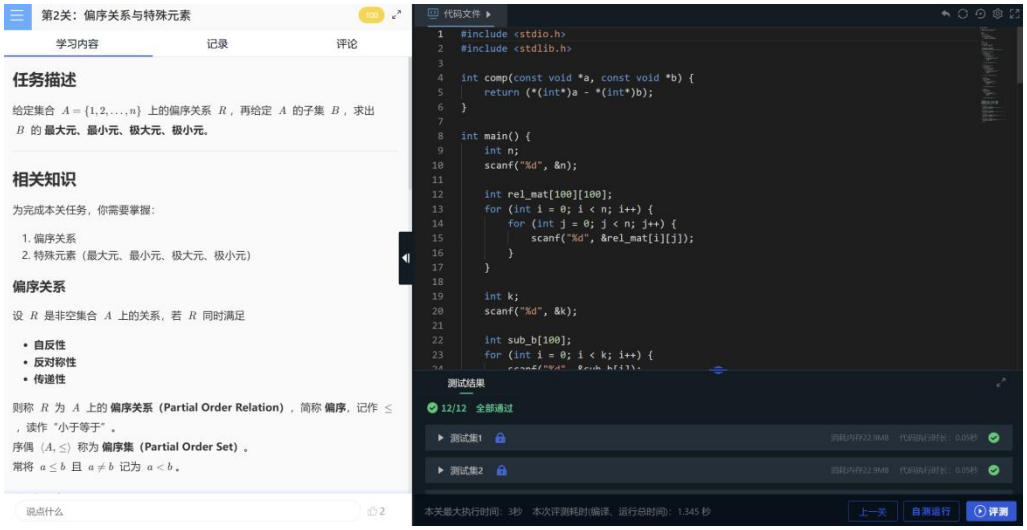
1. 时间复杂度：O（k^2），遍历子集元素两两比较

2. 空间复杂度：O（k），存储子集与四类特殊元素

核心代码片段：

```
// 求极大元
for (int i=0;i<k;i++){
    int b=sub_b[i];
    int is_max=1;
    for(int j=0;j<k;j++){
        int x=sub_b[j];
        if(b==x)continue;
        if(rel_mat[b-1][x-1]==1){
            is_max=0;
            break;
        }
    }
    if(is_max)max_e[max_cnt++]=b;
}
// 求极小元、最大元、最小元逻辑同上
```

截图证明：


第2关：偏序关系与特殊元素
学习内容
记录
评论
任务描述
给定集合 $A = \{1, 2, \ldots, n\}$ 上的偏序关系 R，再给定 A 的子集 B，求出 B 的 最大元、最小元、极大元、极小元。
相关知识
为完成本关任务，你需要掌握：
1. 偏序关系
2. 特殊元素（最大元、最小元、极大元、极小元）
偏序关系
设 R 是非空集合 A 上的关系，若 R 同时满足
自反性
反对称性
传递性
则称 R 为 A 上的 偏序关系（Partial Order Relation），简称 偏序，记作 $\leq$，读作“小于等于”。
序偶 $\langle A, \leq \rangle$ 称为 偏序集（Partial Order Set）。
常将 $a \leq b$ 且 $a \neq b$ 记为 $a < b$。
说点什么
代码文件
#include <stdio.h>
#include <stdlib.h>
int comp(const void *a, const void *b) {
return (*(int*)a - *(int*)b);
}
int main() {
int n;
scanf("%d", &n);
int rel_mat[100][100];
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
scanf("%d", &rel_mat[i][j]);
}
}
int k;
scanf("%d", &k);
int sub_b[100];
for (int i = 0; i < k; i++) {
测试结果
12/12 全部通过
测试集1
测试集2
本关最大执行时间：3秒 本次评测耗时(编译、运行总时间)：1.345 秒
上一关
自测运行
评测

4　函数

4.1 实验目的

（1）掌握函数（映射）的定义判定方法，能够区分关系与函数的差异。

（2）学会判断函数的类型：单射、满射、双射，理解三类函数的核心性质。

（3）实现函数的复合运算、置换运算，掌握有限集合上函数迭代的计算方法。

（4）建立幂集与二进制字符串的双射映射，理解双射在集合计数与编码中的应用。

4.2 实验内容

4.2.1 函数的判断

（1）函数的定义。给定集合 A 到 B 的关系 f，判断 f 是否是 A 到 B 上的函数（映射）。

（2）函数的类型。给定集合 A 与 B，再给定集合 A 到 B 的函数 f，判断 f 是否是 A 到 B 上的单射、满射，或双射。

4.2.2 函数的运算

（1）函数的复合运算与置换。给定集合 A 与 A 上的置换函数 f，求其复合 m 次后的置换。

（2）函数的应用。

设$A_n = \{a_1, a_2, a_3, \ldots, a_n\}$是一个包含$(n)$个元素的有限集合，定义集合

$B_n = \{b_1 b_2 b_3 \ldots b_n | b_i \in \{0,1\}\}$，即所有长度为$(n)$的 01 字符串的集合。

试建立从幂集$(P(A_n))$到(B_n)的一个双射，满足：对任意子集$(S \in P(A_n))$，令$f(S) = b_1 b_2 b_3 \ldots b_n$其中$b_i = \begin{cases} 1, & a_i \in S \\ 0, & a_i \notin S \end{cases}, \quad i = 1,2,\ldots,n$

4.3 实验过程

4.3.1 函数的判断

（1）函数的定义

解题思路：遍历所有关系序偶，检查同一个 x 是否对应多个不同的 y，如果存在则不是函数，否则是函数。

算法复杂度分析：

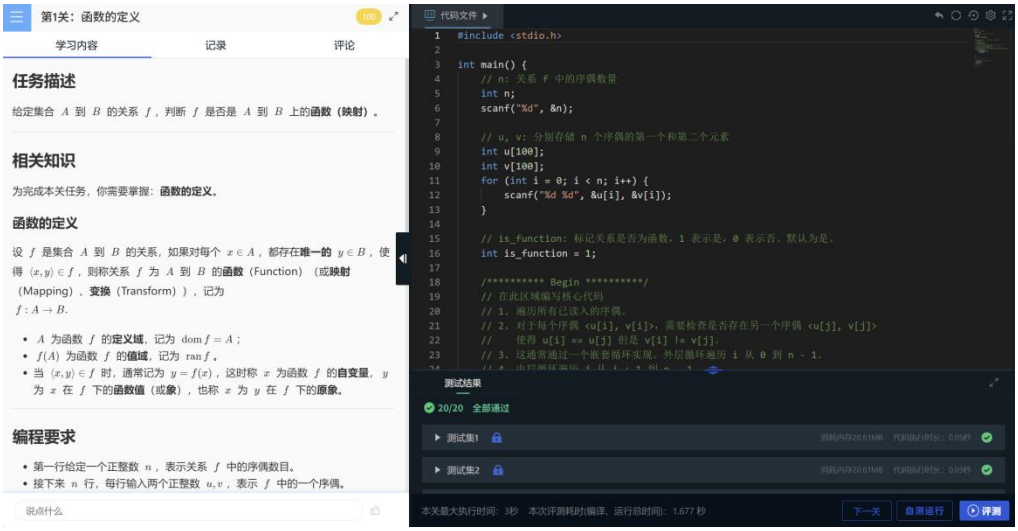
1. 时间复杂度：O（n^2），双层循环检查所有序偶对

2. 空间复杂度：O（n），存储输入的序偶

核心代码片段：

```
// 检查是否存在 x 对应多个 y 的情况
for(int i=0;i<n;i++){
    for(int j=i+1;j<n;j++){
        if(u[i]==u[j]&&v[i]!=v[j]){
            is_function=0;
            goto end;
        }
    }
}
end:
```

截图证明：



（2）函数的类型

解题思路：先判断是否是合法函数，再检查 y 值是否重复判断单射，检查 B 中元素是否全部覆盖判断满射，同时满足为双射。

算法复杂度分析：

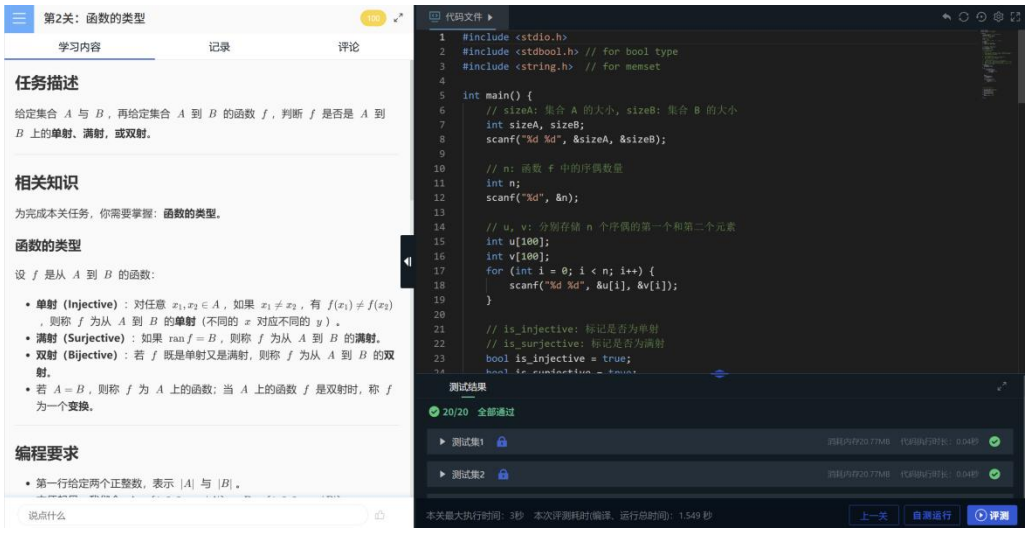
1. 时间复杂度：O（n^2），检查重复与覆盖

2. 空间复杂度：O（sizeB），使用数组标记值域

核心代码片段：

```
// 判断单射：y 不能重复
for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
                if(v[i]==v[j]){
                        is_injective=false;
                        goto step2;
                }
        }
}
step2:
// 判断满射：B 中所有元素必须出现
int a[sizeB+1]={0};
for(int i=0;i<n;i++) a[v[i]]=1;
for(int i=1;i<=sizeB;i++){
        if(a[i]==0) is_surjective=false;
}
```

截图证明：



4.3.2 函数的运算

（1）函数的复合运算与置换

解题思路：初始化恒等置换，循环 m 次执行复合运算，每次计算 new_res(i)=f（res_map（i）），用临时数组保存结果并更新。

算法复杂度分析：

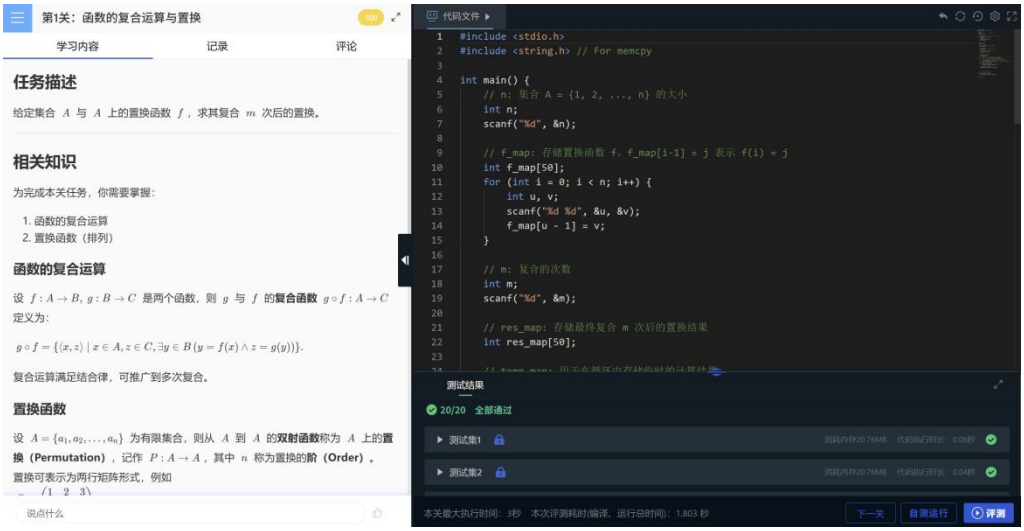
1. 时间复杂度：O（m*n），m 次循环，每次遍历 n 个元素

2. 空间复杂度：O（n），存储置换与临时结果

核心代码片段：

```
// 迭代 m 次求复合置换
for(int t=0;t<m;t++){
    for(int i=0;i<n;i++){
        temp_map[i]=f_map[res_map[i]-1];
    }
    memcpy(res_map,temp_map,sizeof(res_map));
}
```

截图证明：



（2）函数的应用

解题思路：用二进制 mask 枚举所有子集，每一位对应元素是否存在，输出子集与对应 01 串，实现幂集到 01 串集合的双射。

算法复杂度分析：

1. 时间复杂度：O（$n*2^n$），遍历 2^n 个子集，每个检查 n 位

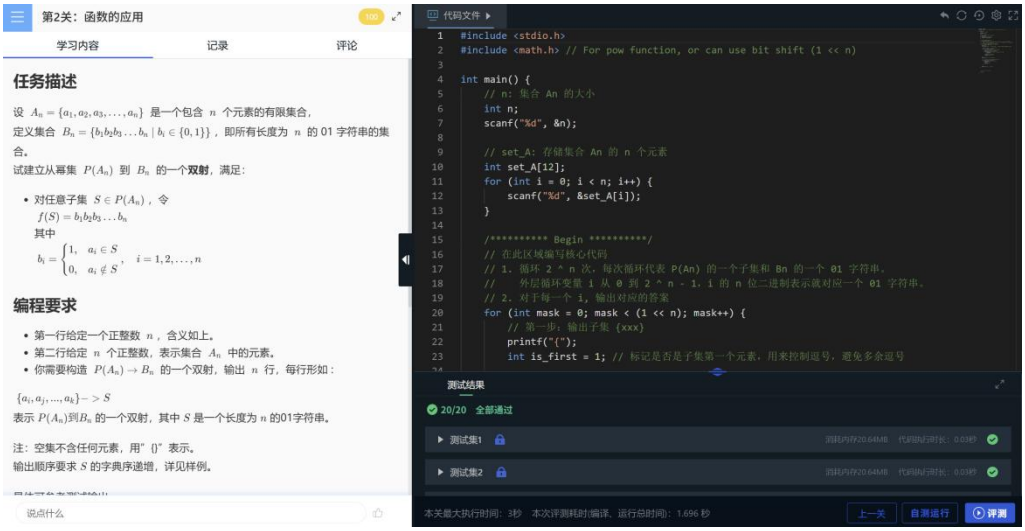
2. 空间复杂度：O（n），存储集合元素

核心代码片段：

```
// 二进制 mask 生成所有子集与 01 串
for(int mask=0;mask<(1<<n);mask++){
    // 输出子集
    printf("{");
    for(int j=0;j<n;j++){
        if(mask & (1 << (n-1-j))){
            if(!is_first) printf(",");
            printf("%d",set_A[j]);
            is_first=0;
```

```
            }
        }
        printf("}->");
        // 输出 01 串
        for(int j=0;j<n;j++){
            if(mask & (1 << (n-1-j))) printf("1");
            else printf("0");
        }
        printf("\n");
    }
```

截图证明：



5　初等数论

5.1 实验目的

（1）掌握大整数模运算、快速幂（二进制模幂）算法，解决大数计算溢出问题。

（2）实现素数判定（试除法）、整数素因数分解、最大公约数求解（扩展欧几里得算法）。

（3）掌握乘法逆元、线性同余方程、中国剩余定理的求解方法，理解同余理论的核心应用。

（4）实现经典密码算法：仿射密码、RSA 密钥生成与加密，理解数论在现代密码学中的基础作用。

（5）提升大数处理、模运算优化的编程能力，为密码学与信息安全打下基础。

5.2 实验内容

5.2.1 除法和模运算

（1）判断是否同余。本关任务：编写一个程序，计算两个大整数（最多可达 10^5位）是否模 m 同余。

（2）二进制模幂算法。本关任务：编写一个快速计算一个数在模运算下的若干次方的程序。例如，计算 $a^n(\mathrm{mod}m)$。

5.2.2 素数和最大公因数

（1）实现试除法判断素数。本关任务：编写一个实现试除法判断素数的程序。

（2）唯一分解定理。本关任务：找出给定整数的素因数分解

（3）扩展欧几里得算法。本关任务：找到最大公约数（GCD）的一组线性表示，即找到整数 x,y 使得 gcd(a,b)=ax+by。

5.2.3 同余方程

（1）求一个整数模 m 的逆元。本关任务：编写一个程序，求一个整数 a 模 m 的乘法逆元。

（2）求解线性同余方程 ax ≡ b (mod m)。本关任务：编写一个程序，求解形如 ax≡b(modm) 的线性同余方程。

说明：题目保证 a 和 m 互质，因此可以直接利用乘法逆元求解。

（3）使用中国剩余定理求解同余方程组。本关任务：编写一个程序，使用中国剩余定理 (CRT) 求解一元线性同余方程组。

该方程组形如：

$$\begin{cases} x \equiv a_1 & mod m_1 \\ x \equiv a_2 & mod m_2 \\ \vdots & \\ x \equiv a_n & mod m_n \end{cases}$$

5.2.4 密码学

（1）仿射密码。本关任务：编写一个程序来实现仿射密码的加密过程。仿射密码是移位密码（凯撒密码）的一种泛化。凯撒密码的加密函数是 C=(P+k)(mod26)，而仿射密码采用了更一般的线性函数：C=(a· P+b)(mod26)。

（2）寻找私钥。本关任务：编写一个程序来模拟 RSA 密码系统中私钥的生成。RSA 是一种应用最广泛的公钥密码系统。它使用一对密钥：

公钥 (n,e)：用于加密，可以公开给任何人。

私钥 (n,d)：用于解密，必须由持有者严格保密。

本关的任务就是根据生成公钥的参数，反向计算出对应的私钥 d。

（3）RSA 加密。本关任务：编写一个程序来实现 RSA 密码系统的加密过程。RSA 加密是公钥密码学的核心操作。持有者公开其公钥 (n,e)，任何人都可以使用这个公钥将明文消息 M 加密为密文 C。

5.3 实验过程

5.3.1 除法和模运算

（1）判断是否同余

解题思路：对超长数字字符串逐位取模，避免溢出，分别计算两个大数对 m 的余数，余数相同则同余。

算法复杂度分析：

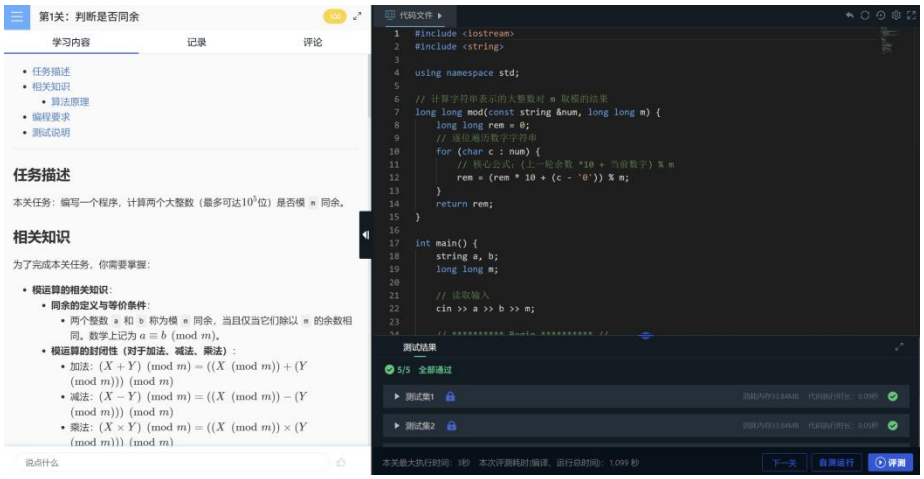
1. 时间复杂度：O（len），逐位遍历数字字符串

2. 空间复杂度：O（1），仅使用少量变量

核心代码片段：

```
// 逐位计算大整数模 m
long long mod(const string &num, long long m) {
    long long rem = 0;
    for (char c : num) {
        rem = (rem * 10 + (c - '0')) % m;
    }
    return rem;
}
// 比较余数是否相等
if (rem_a == rem_b) cout << "yes" << endl;
else cout << "no" << endl;
```

截图证明：



（2）二进制模幂算法

解题思路：使用快速幂，按二进制分解指数，每次平方底数，奇数位乘入结果，全程取模防止溢出。

算法复杂度分析：

1. 时间复杂度：O（log n），指数按二进制折半计算

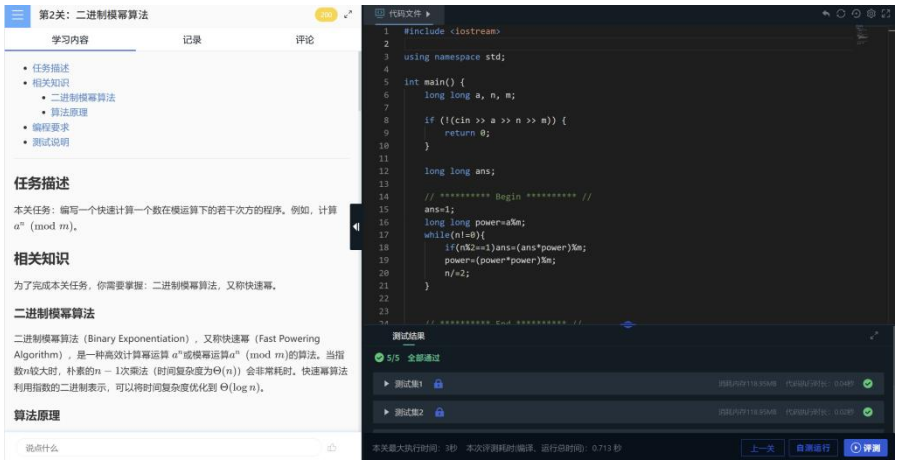
2. 空间复杂度：O（1），仅使用临时变量

核心代码片段：

```
// 快速幂求 a^n mod m
ans = 1;
long long power = a % m;
```

```
while(n != 0) {
    if(n % 2 == 1) ans = (ans * power) % m;
    power = (power * power) % m;
    n /= 2;
}
```

截图证明：



5.3.2 素数和最大公因数

（1）实现试除法判断素数

解题思路：小于 2 非素数，2、3 是素数，遍历 2 到 sqrt(n)，能整除则非素数，否则是素数。

算法复杂度分析：

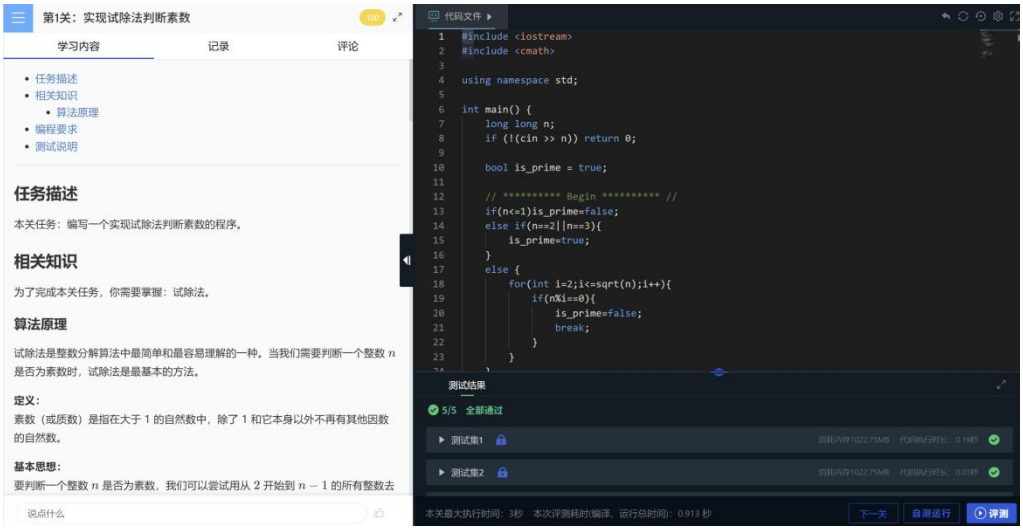
1. 时间复杂度：O（sqrt（n)），遍历到根号 n

2. 空间复杂度：O（1)，无额外数组

核心代码片段：

```
if(n <= 1) {
    is_prime = false;            // 小于等于 1 不是素数
} else if(n == 2 || n == 3) {
    is_prime = true;             // 2 和 3 是素数
} else {
    // 遍历 2 到 sqrt(n)
    for(int i = 2; i <= sqrt(n); i++) {
        if(n % i == 0) {         // 能整除，不是素数
            is_prime = false;
            break;
        }
    }
}
```

截图证明：



（2）唯一分解定理

解题思路：从 2 开始试除，统计每个素因子的出现次数，保存素数与指数，输出分解结果。

算法复杂度分析：

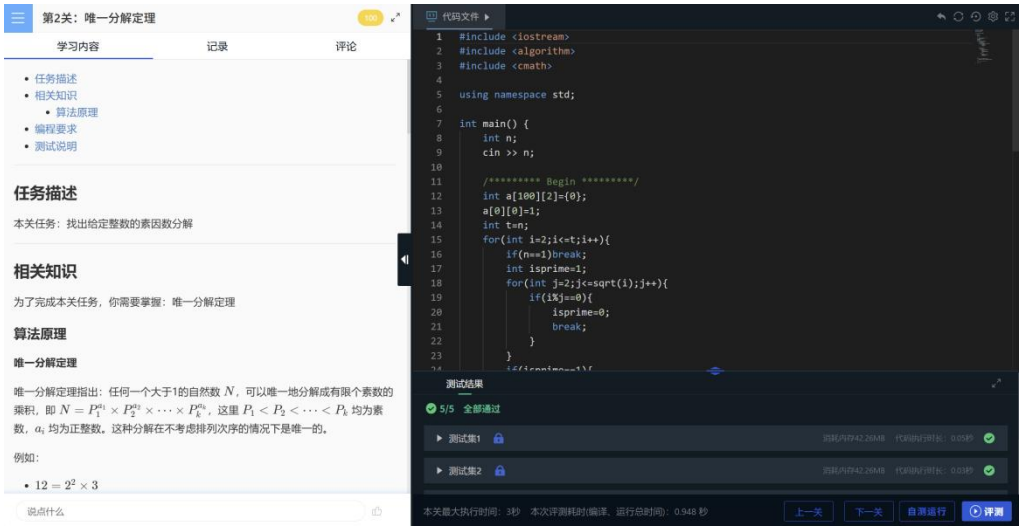
1. 时间复杂度：O（sqrt（n）），试除到根号 n

2. 空间复杂度：O（因子数），存储素因子与指数

核心代码片段：

```
for(int i=2;i<=t;i++){
    if(n==1)break;
    int isprime=1;
    for(int j=2;j<=sqrt(i);j++)
        if(i%j==0) isprime=0; // 判定 i 是否为素数
    if(isprime==1){
        while(n>1&&n%i==0){ // 反复除尽当前素因子
            n/=i;
            a[a[0][0]][1]++; // 指数累加
        }
        if(a[a[0][0]][1]>0){
            a[a[0][0]][0]=i;
            a[0][0]++;
        }
    }
}
```

截图证明：



（3）扩展欧几里得算法

解题思路：在传统欧几里得求最大公约数的基础上递归拓展，递归终止条件为 b 等于 0，此时初始化系数 x=1、y=0；逐层回溯递归结果，通过回代公式更新上层 x、y 的值，最终得到满足 ax+by=gcd(a,b) 的一组整数解 x、y，同时返回 a 与 b 的最大公约数。

算法复杂度分析：

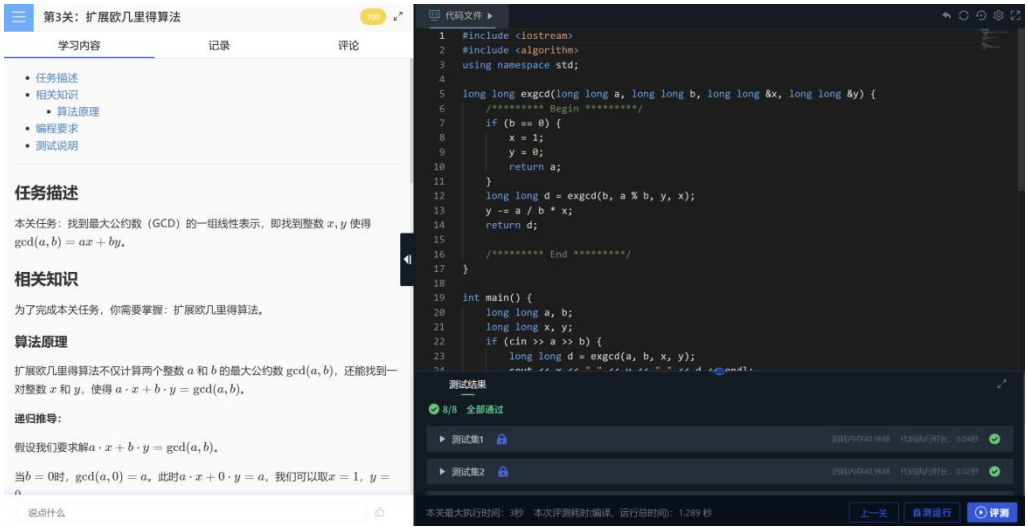
1.时间复杂度：O(log min(a,b))，递归过程数据不断取模快速缩小数值

2.空间复杂度：O(log min(a,b))，递归调用产生栈空间开销

核心代码片段：

```
long long exgcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1; y = 0; return a; // 递归边界
    }
    long long d = exgcd(b, a % b, y, x);
    y -= a / b * x; // 回代修正 y 系数
    return d;
}
```

截图证明：



5.3.3 同余方程

（1）求一个整数模 m 的逆元

解题思路：乘法逆元存在的充要条件是 a 与模数 m 互质，借助扩展欧几里得算法求解 ax+my=1 的整数解 x；若 gcd(a,m) 不等于 1，二者不互质无乘法逆元，返回 -1；若互质，将解 x 对 m 取模并修正为正数，得到 a 模 m 的最小正乘法逆元。

算法复杂度分析：

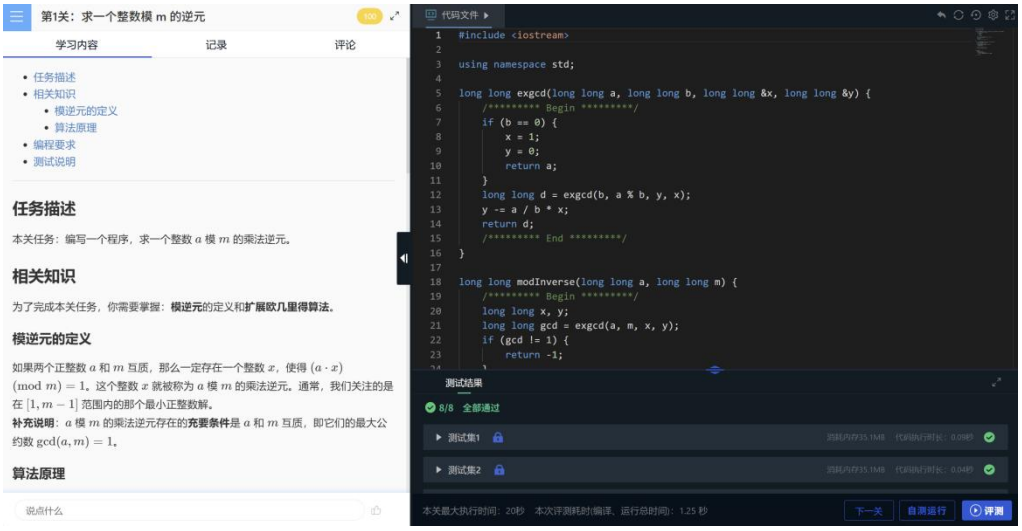
1. 时间复杂度：O(log m)，运算耗时由扩展欧几里得算法决定

2. 空间复杂度：O(1)，仅临时存储求解系数 x、y

核心代码片段：

```
long long modInverse(long long a, long long m) {
    long long x, y;
    long long gcd = exgcd(a, m, x, y);
    if (gcd != 1) return -1; // 不互质无逆元
    return (x % m + m) % m;   // 规范化为正逆元
}
```

截图证明：



（2）求解线性同余方程 ax ≡ b (mod m)

解题思路：题目限定参数 a 和 m 互质，因此 a 模 m 一定存在唯一乘法逆元；根据同余方程变形可得 x ≡ inv (a)*b (mod m)，先调用逆元函数算出 a 的模 m 逆元，逆元与 b 相乘后对 m 取模，结果即为该线性同余方程的最小正整数解。

算法复杂度分析：

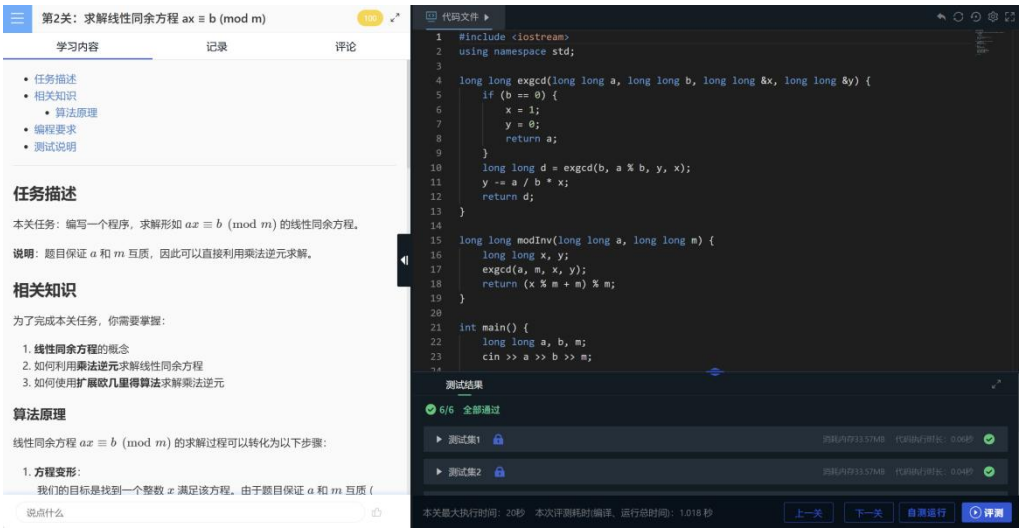
1. 时间复杂度：O (log m)，运算开销集中在逆元求解步骤

2. 空间复杂度：O (1)，常数变量保存中间结果

核心代码片段：

```
long long inv = modInv(a, m);
long long ans = (inv * b) % m; // 代入公式计算方程解
```

截图证明：



（3）使用中国剩余定理求解同余方程组

解题思路：中国剩余定理适用前提为所有模数两两互质，先计算全部模数的累积乘积 M;逐个拆分方程组,令 Mi=M/mi,求出 Mi 在模 mi 下的乘法逆元 ti，单项贡献为 $a_i M_i t_i$，累加所有单项后对 M 取模，最终结果就是整个同余方程组在模 M 下的唯一最小正整数解。

算法复杂度分析：

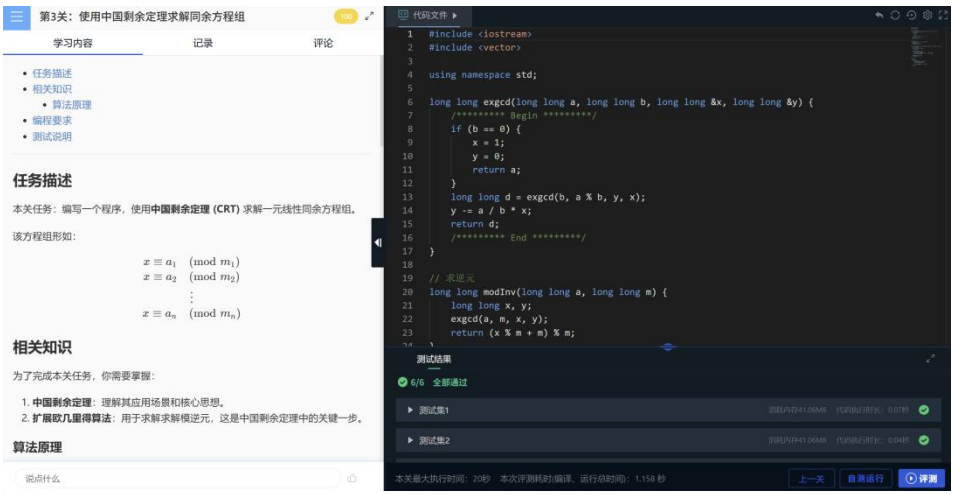
1.时间复杂度：O（n log m），n 为方程条数，每条方程求逆耗时 log m

2.空间复杂度：O（n），数组存储每组余数 a 与模数 m

核心代码片段：

```
long long M = 1;
for(int i=0;i<n;i++) M *= m[i]; // 计算所有模数总乘积
long long ans=0;
for(int i=0;i<n;i++){
        long long Mi = M/m[i];
        long long ti = modInv(Mi,m[i]); // 求 Mi 模 mi 逆元
        ans += a[i]*Mi*ti;
}
ans = (ans%M+M)%M; // 修正为最小正解
```

截图证明：



5.3.4 密码学

（1）仿射密码

解题思路：仿射加密公式为 C=(a·P+b) mod 26，仅对大写英文字母加密，将大写字符减去 'A' 转换为 0~25 的数字 P,代入加密公式运算得到密文数值，再加 'A' 转回大写字母；非大写字母保持原有字符不变，最终拼接得到完整密文字符串。

算法复杂度分析：

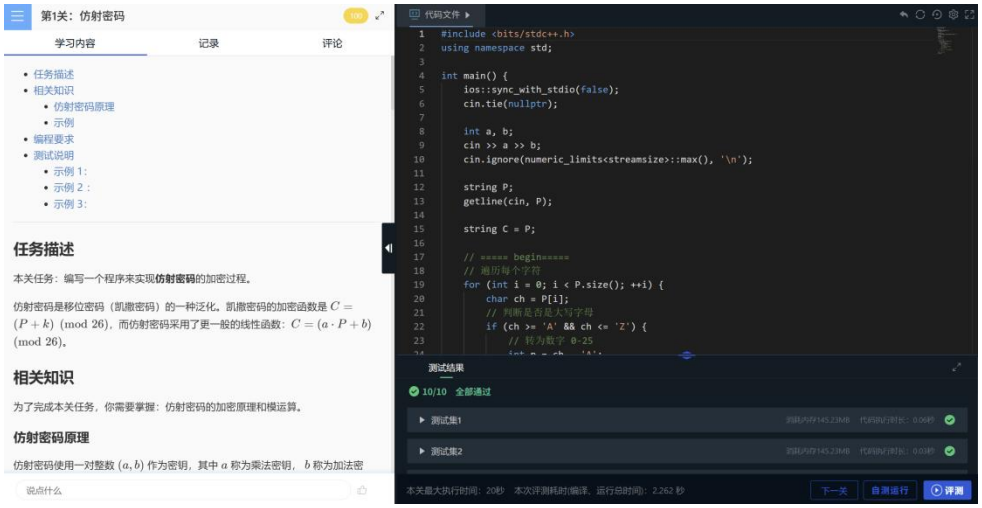
1.时间复杂度：O（len），len 代表明文字符串长度，遍历一次明文

2.空间复杂度：O（len），需要开辟空间存储密文结果

核心代码片段：

```
int p = ch - 'A';
int c = (a * p + b) % 26; // 仿射加密计算公式
C[i] = 'A' + c;
```

截图证明：



（2）寻找私钥

解题思路：RSA 密钥生成规则中，已知两个素数 p、q，欧拉函数 phi=(p-1)*(q-1)；公钥 e 与 phi 互质，私钥 d 是 e 在模 phi 下的乘法逆元，通过扩展欧几里得求解逆元，最后将结果规范化为正整数，得到 RSA 解密私钥 d。

算法复杂度分析：

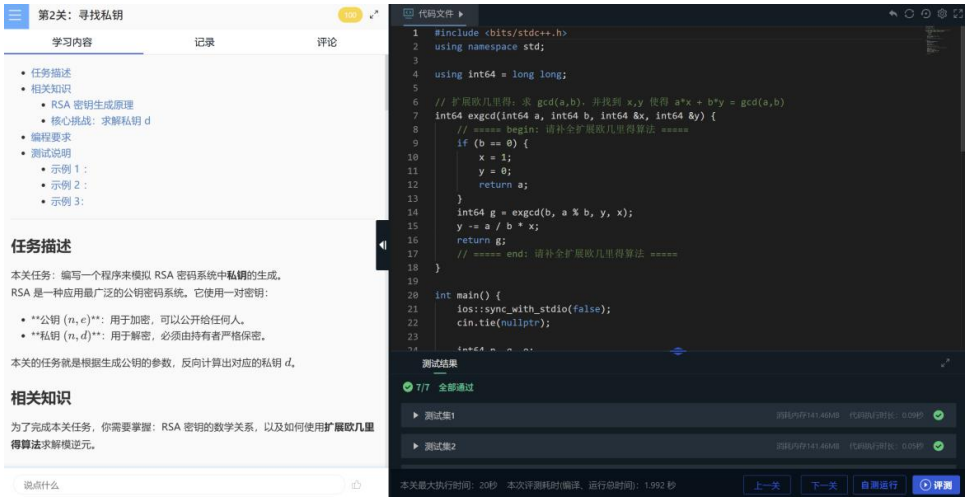
1.时间复杂度：O（log phi），时间消耗来自扩展欧几里得求逆

2.空间复杂度：O（1），只用临时变量存储系数与中间值

核心代码片段：

```
int64 phi = (p-1)*(q-1); // 计算欧拉函数值
exgcd(e, phi, x, y);
int64 d = (x%phi+phi)%phi; // 规范得到私钥 d
```

截图证明：



（3）RSA 加密

解题思路：RSA 加密公式 C=M^e mod n，利用快速幂算法完成大指数模运算，明文 M 预先对 n 取模缩小数值；循环拆分指数二进制，底数持续平方取模，指数位为 1 时乘入结果并取模，最终得到密文 C。

算法复杂度分析：

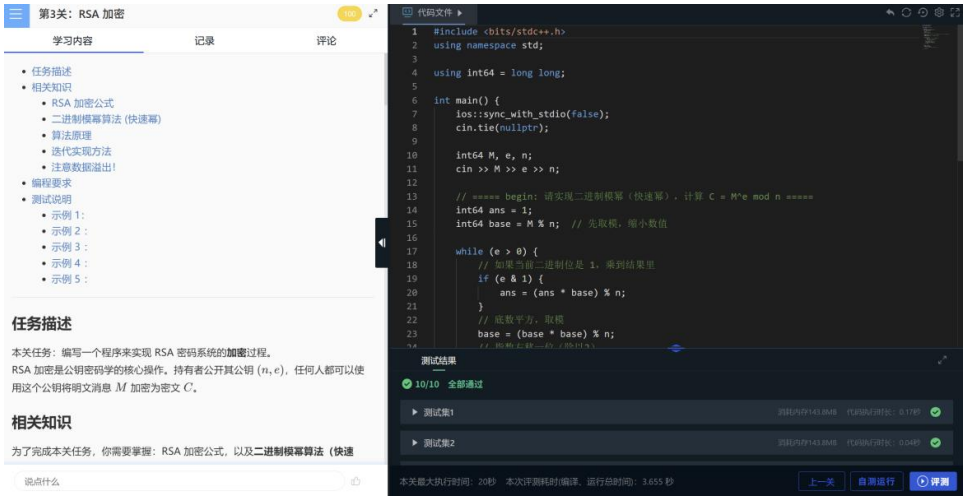
1. 时间复杂度：O（log e），快速幂循环次数等于指数 e 二进制位数

2. 空间复杂度：O（1），常数变量存储底数与密文结果

核心代码片段：

```
int64 ans=1;
int64 base = M%n; // 明文预先取模
while(e>0){
    if(e&1) ans=(ans*base)%n; // 二进制位为 1，累乘结果
    base=(base*base)%n;          // 底数平方取模
    e>>=1;
}
```

截图证明：



6　组合数学

6.1 实验目的

（1）掌握加法法则、乘法法则、可重复组合、多重集排列等基础计数方法，解决实际组合计数问题。

（2）应用鸽巢原理解决存在性证明问题（果实采摘、子集和、平面整中点）。

（3）理解动态规划在带约束计数问题中的应用，实现合法密码数量求解。

（4）掌握生成函数的卷积思想，解决魔力值组合计数、特殊字符串构造计数问题。

（5）建立组合数学建模能力，能够将实际问题转化为标准组合模型求解。

6.2 实验内容

6.2.1 计数基础

（1）网站用户名。本关任务：应用组合数学中的加法法则和乘法法则，计算一个网站所有可能的合法用户名总数。

题目背景：一个网站的用户名设计规则如下：

规则一：可以由 4 个小写英文字母组成。

规则二：或者，可以由 1 个大写英文字母开头，后跟 2 个数字组成。

请问总共有多少种合法的用户名？

（2）相邻的约束。本关任务：你需要构造一个长度为 N 的密码，密码的每一位都是 0-9 的数字。但是，存在 K 对“不吉利”的相邻数字。例如，若(1, 3)是不吉利的，则密码中不允许出现’1’后面紧跟着’3’的情况。请你计算可以构造出多少种合法的 N 位密码。由于结果可能很大，请对 10^9+7 取模。

6.2.2 鸽巢原理

（1）采摘果实。本关任务：应用广义鸽巢原理解决一个问题。

题目背景：果园里有 K 种果树，每种果树上的果实都取之不尽。为了保证你采摘的果实中，至少有 M 个是属于同一种类的，你最少需要采摘多少个果实？

（2）子集和。本关任务：利用鸽巢原理和前缀和技巧，解决“子集和”。

题目背景：给定一个包含 N 个正整数的数组 A。请找出一个非空子集，使得该子集中所有元素的和能被 N 整除。如果存在多个解，输出任意一个即可。题目保证有解。

（3）平面上的整中点。本关任务：利用鸽巢原理，解决“平面上的整中点”问题。

题目背景：在二维平面上给定 N 个坐标都是整数的点。请找出一对点，使得它们的中点坐标也都是整数。如果存在多组解，输出任意一组即可。题目保证N≥5。

6.2.3 排列与组合

（1）饼干选购。本关任务：应用可重复组合的知识，解决“饼干选购”问题。

题目背景：一家饼干店有 N 种不同口味的饼干，每种口味的饼干数量都足够多。你需要购买 K 块饼干，问总共有多少种不同的购买方案？（不考虑选取饼干的顺序）

（2）单词重排。本关任务：应用组合数学中带有多重集的排列知识，解决“单词重排”问题。

题目背景：给定一个可能包含重复字母的字符串（例如 "SUCCESS"），计算重新排列这个字符串中的所有字母可以组成多少个不同的新字符串。

6.2.4 生成函数

（1）游戏。本关任务：小明和小红是两个喜欢收集奇特石头的朋友。小明有一个袋子，里面装着 N 块石头，每块石头上都标有一个“魔力值”。小红也有一个袋子，里面装着 M 块石头，同样每块石头有自己的魔力值。

现在，他们俩要玩一个游戏：各自从自己的袋子里随机取出一块石头。一个组合的“总魔力值”等于两人取出的石头魔力值之和。

请你计算，所有可能的“总魔力值”及其出现的组合数。

（2）特殊的字符串。本关任务：你需要使用字符 'A','B','C' 来构造一个长度为 N 的字符串，要求构造出的字符串中：

'A' 必须出现偶数次。

'B' 必须出现奇数次。

’C’ 的出现次数没有限制。

请计算总共有多少种满足条件的字符串。由于结果可能很大，请对 10^9+7 取模。

6.3 实验过程

6.3.1 计数基础

（1）网站用户名

解题思路：应用加法法则，合法用户名总数等于两种规则的方案数之和。规则一为 4 个小写字母，每个位置有 26 种选择，共 26^4 种；规则二为 1 个大写字母开头后跟 2 个数字，大写字母 26 种、每个数字 10 种，共 261010 种，两者相加得到总方案数。

算法复杂度分析：

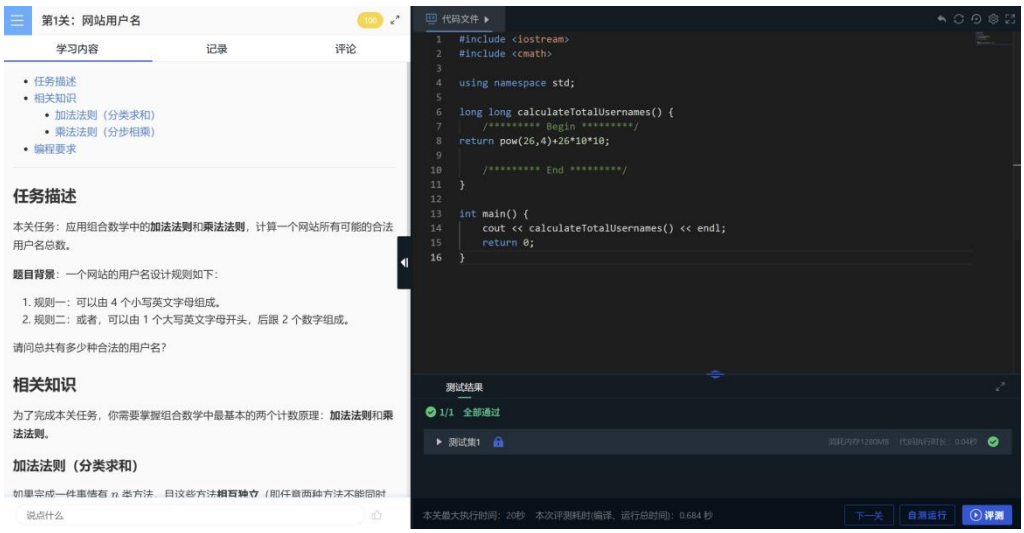
1. 时间复杂度：O（1），仅进行固定次数的算术运算

2. 空间复杂度：O（1），无额外变量开销

核心代码片段：

```
// 加法法则:两种规则方案数相加
return pow(26,4)+26*10*10;
```

截图证明：



（2）相邻的约束

解题思路： 采用动态规划求解，定义 dp[i][j] 为长度为 i、最后一位是 j 的合法密码数。初始状态长度为 1 时，每个数字都有 1 种方案；状态转移时，对于每个当前位数字，累加所有不被禁止的上一位数字的方案数；最终结果为长

度 n 时所有最后一位方案数之和，全程对 10^9+7 取模。

算法复杂度分析：

1. 时间复杂度：O（n），外层循环 n 次，内层固定 10*10 次循环

2. 空间复杂度：O（n），存储 n+1 行 10 列的 DP 数组

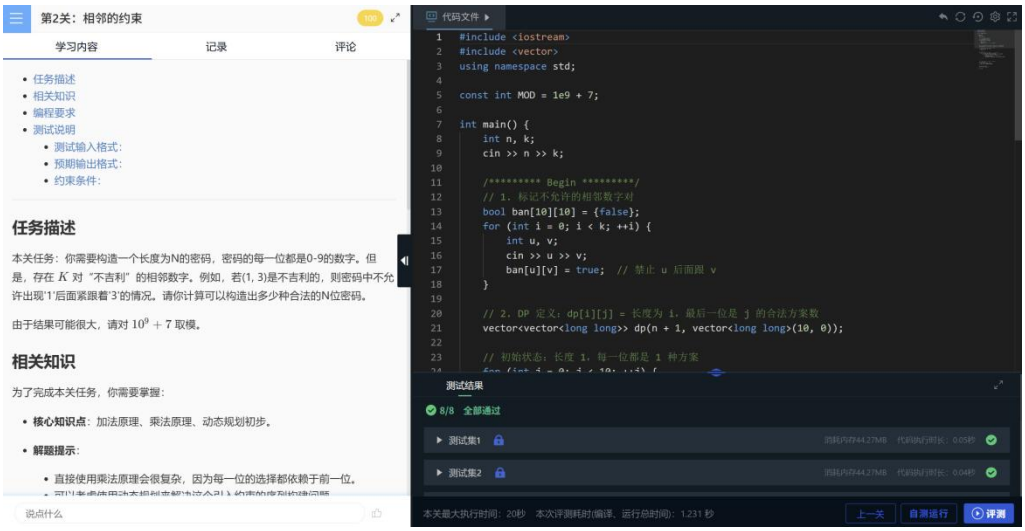
核心代码片段：

```
// 标记禁止的相邻数字对
bool ban[10][10] = {false};
for (int i = 0; i < k; ++i) {
        int u, v;
        cin >> u >> v;
        ban[u][v] = true;
}

// DP 初始化：长度 1 时每个数字 1 种方案
vector<vector<long long>> dp(n + 1, vector<long long>(10, 0));
for (int j = 0; j < 10; ++j) dp[1][j] = 1;

// 状态转移
for (int i = 2; i <= n; ++i) {
        for (int last = 0; last < 10; ++last) {
                for (int now = 0; now < 10; ++now) {
                        if (!ban[last][now]) { // 相邻不禁止则转移
                                dp[i][now] = (dp[i][now] + dp[i-1][last]) % MOD;
                        }
                }
        }
}
```

截图证明：



6.3.2 鸽巢原理

（1）采摘果实

解题思路：应用广义鸽巢原理，最坏情况下每种果树都采摘了 m-1 个果实，此时再采摘 1 个，必然有至少 m 个果实属于同一种类，因此最少采摘数量为 k*(m-1)+1。

算法复杂度分析：

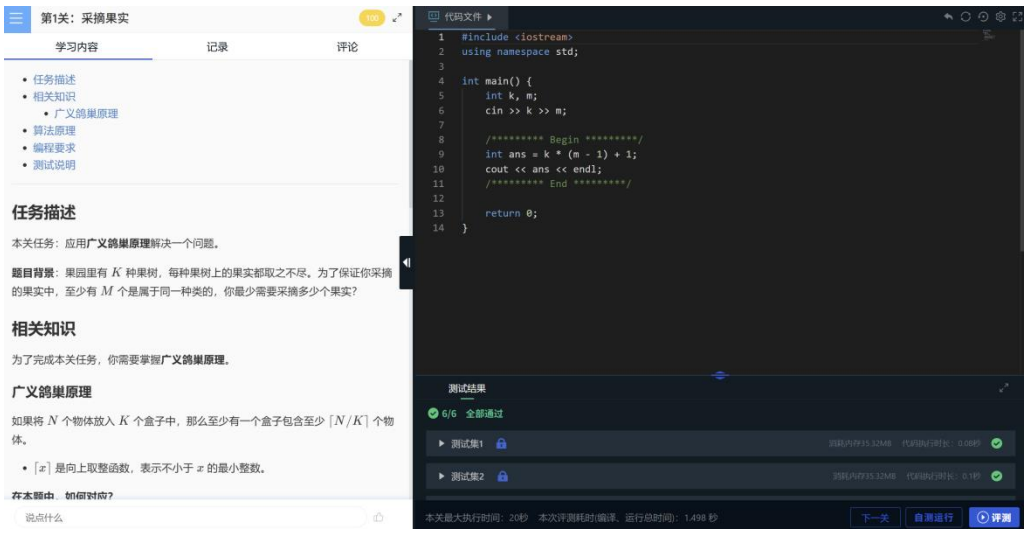
1. 时间复杂度：O（1），仅一次算术运算

2. 空间复杂度：O（1），无额外空间开销

核心代码片段：

```
// 广义鸽巢原理公式
int ans = k * (m - 1) + 1;
```

截图证明：



（2）子集和

解题思路：利用前缀和与鸽巢原理，n 个元素有 n+1 个前缀和（包括 0），模 n 后必有两个前缀和余数相同，它们的差就是能被 n 整除的子集。计算前缀和并记录每个余数第一次出现的位置，若余数为 0 则直接输出前 i+1 个元素；若找到重复余数则输出两个位置之间的元素。

算法复杂度分析：

1. 时间复杂度：O（n），遍历数组一次计算前缀和

2. 空间复杂度：O（n），存储前缀和余数的位置数组

核心代码片段：

```
vector<int> pos(n, -1); // 记录余数第一次出现的下标
```

```
long long sum = 0;

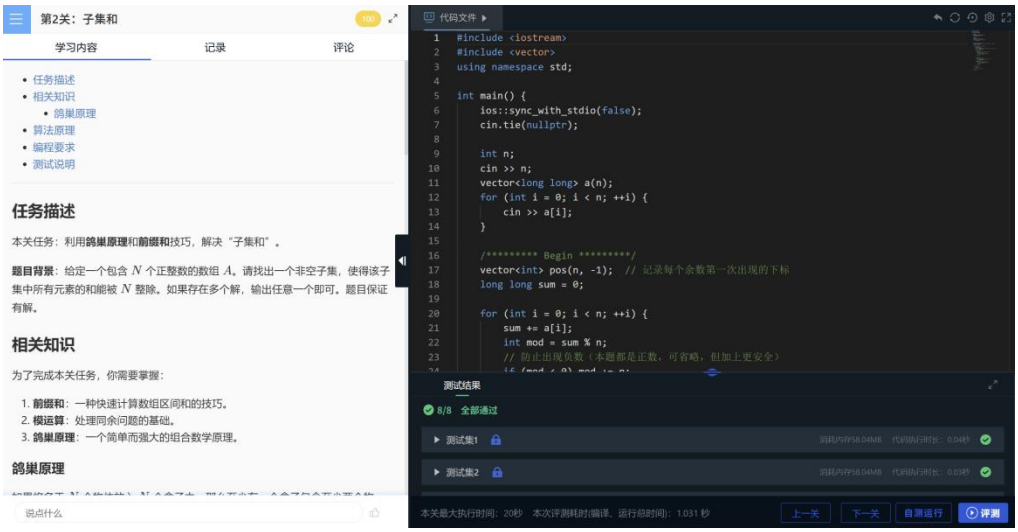
for (int i = 0; i < n; ++i) {
        sum += a[i];
        int mod = sum % n;
        if (mod < 0) mod += n; // 处理负数余数

        if (mod == 0) { // 前缀和直接能被 n 整除
                cout << i + 1 << '\n';
                for (int j = 0; j <= i; ++j) cout << a[j] << ' ';
                return 0;
        }

        if (pos[mod] != -1) { // 找到重复余数，输出中间子集
                int l = pos[mod] + 1;
                int r = i;
                cout << r - l + 1 << '\n';
                for (int j = l; j <= r; ++j) cout << a[j] << ' ';
                return 0;
        }

        pos[mod] = i; // 记录当前余数位置
}
```

截图证明：



（3）平面上的整中点

解题思路：两个点的中点为整数当且仅当它们的 x 坐标和 y 坐标奇偶性都相同。坐标奇偶性共有 4 种组合（(偶，偶)、(偶，奇)、(奇，偶)、(奇，奇)），n≥5 时根据鸽巢原理必有两个点属于同一种组合，它们的中点就是整数。记录

每种类型第一次出现的点，找到同类型时输出。

算法复杂度分析：

1. 时间复杂度：O（n），遍历所有点一次

2. 空间复杂度：O（1），仅用固定大小的数组记录 4 种类型

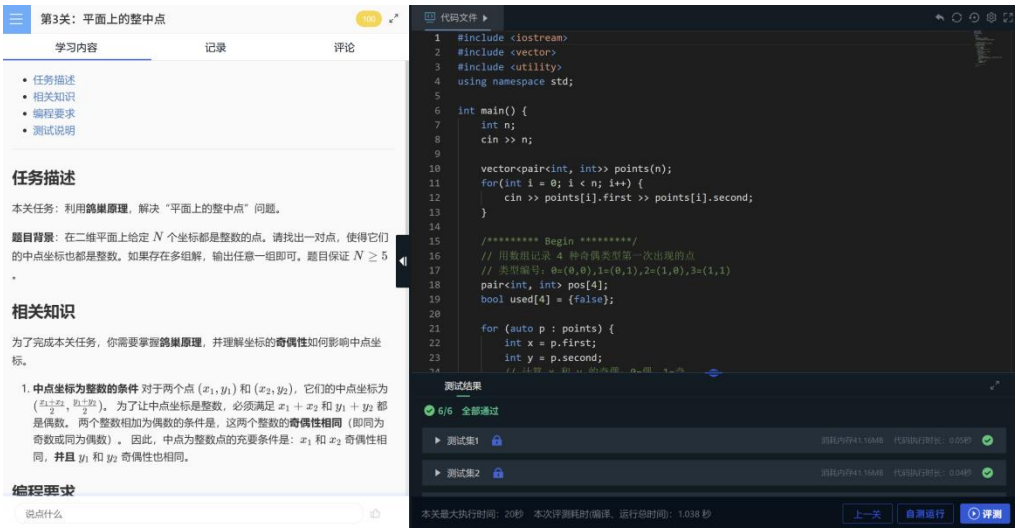
核心代码片段：

```
pair<int, int> pos[4]; // 记录 4 种奇偶类型第一次出现的点
bool used[4] = {false};

for (auto p : points) {
        int x = p.first;
        int y = p.second;
        int tx = x % 2;
        int ty = y % 2;
        if (tx < 0) tx += 2; // 处理负数坐标
        if (ty < 0) ty += 2;
        int type = tx * 2 + ty; // 计算类型编号

        if (used[type]) { // 找到同类型点，输出结果
                cout << pos[type].first << " " << pos[type].second << endl;
                cout << x << " " << y << endl;
                return 0;
        } else {
                used[type] = true;
                pos[type] = p; // 记录第一次出现的点
        }
}
```

截图证明：



6.3.3 排列与组合

（1）饼干选购

解题思路：可重复组合问题，n 种饼干选 k 块的方案数等于组合数 C(n+k-1, k)，即从 n+k-1 个位置中选 k 个位置放隔板。计算时先计算分子(n+k-1)!，再除以分母 k!*(n-1)！得到结果。

算法复杂度分析：

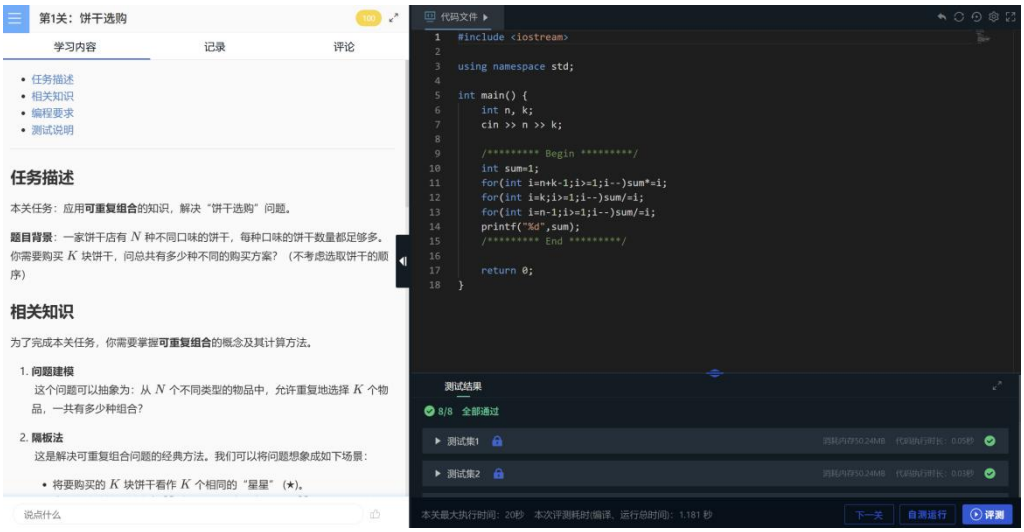
1.时间复杂度：O（(n+k)，计算阶乘的循环次数为 n+k

2.空间复杂度：O（(1)，仅用临时变量存储乘积

核心代码片段：

```
// 计算组合数 C(n+k-1, k)
int sum=1;
for(int i=n+k-1;i>=1;i--)sum*=i; // 分子:(n+k-1)!
for(int i=k;i>=1;i--)sum/=i;        // 除以 k!
for(int i=n-1;i>=1;i--)sum/=i;     // 除以(n-1)!
```

截图证明：



（2）单词重排

解题思路：多重集排列问题，总排列数等于字符串长度的阶乘除以每个重复字母阶乘的乘积。先统计每个字母的出现次数，计算总阶乘，再依次除以每个字母的阶乘，得到不同的字符串数量。

算法复杂度分析：

1.时间复杂度：O（(n)，遍历字符串统计次数，计算阶乘的循环次数为 n

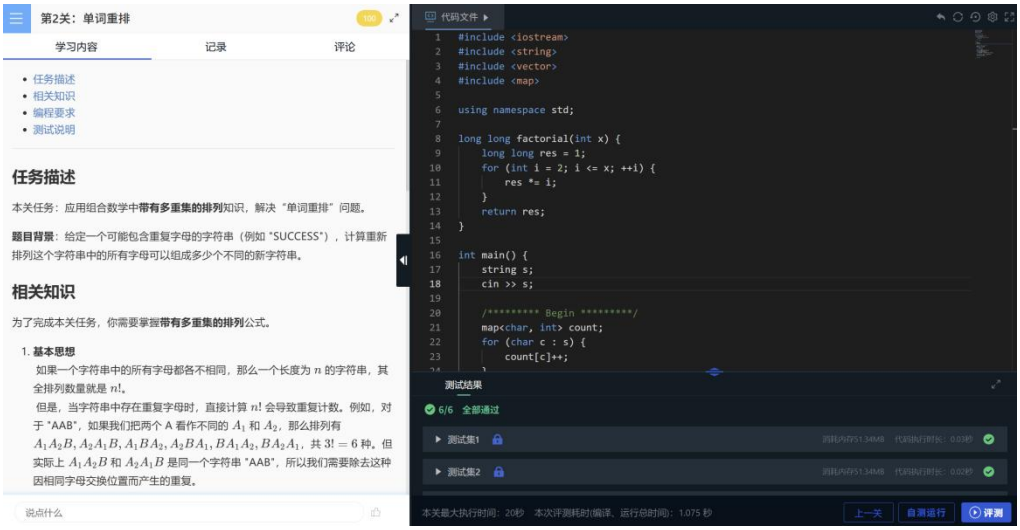
2.空间复杂度：O（(1)，仅用 map 存储字母出现次数（最多 26 个字母）

核心代码片段：

```
map<char, int> count;
for (char c : s) count[c]++; // 统计每个字母出现次数

int n = s.size();
long long total = factorial(n); // 总阶乘
long long denominator = 1;
for (auto& pair : count) {
        denominator *= factorial(pair.second); // 分母：各字母阶乘乘积
}
long long ans = total / denominator;
```

截图证明：



6.3.4 生成函数

（1）游戏

解题思路：生成函数的卷积应用，小明和小红的石头魔力值分别对应两个生成函数，它们的乘积就是总魔力值的生成函数，系数为对应和的出现次数。统计两人魔力值的频次，然后通过双重循环计算所有可能的和及其次数，最后按升序输出。

算法复杂度分析：

1. 时间复杂度：O（1），魔力值最大为 500，双重循环固定 501*501 次

2. 空间复杂度：O（1），仅用固定大小的数组存储频次

核心代码片段：

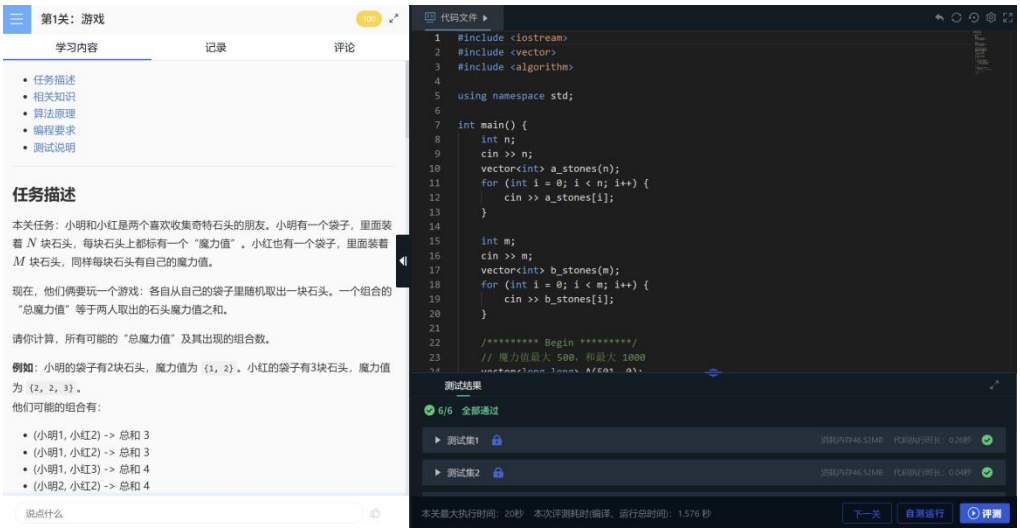
```
// 统计两人魔力值频次
vector<long long> A(501, 0);
vector<long long> B(501, 0);
```

```
for (int x : a_stones) A[x]++;
for (int x : b_stones) B[x]++;

// 卷积计算总魔力值频次
vector<long long> C(1001, 0);
for (int i = 0; i <= 500; ++i) {
    if (A[i] == 0) continue;
    for (int j = 0; j <= 500; ++j) {
        if (B[j] == 0) continue;
        C[i + j] += A[i] * B[j];
    }
}

// 按升序输出结果
for (int s = 0; s <= 1000; ++s) {
    if (C[s] > 0) cout << s << " " << C[s] << endl;
}
```

截图证明：



（2）特殊的字符串

解题思路：通过生成函数推导得到公式：总方案数 =(3^n - (-1)^n)/4 mod MOD。使用快速幂计算 3^n，根据 n 的奇偶性处理 (-1)^n，乘以 4 在模 1e9+7 下的逆元 250000002，得到最终结果。

算法复杂度分析：

1.时间复杂度：时间复杂度：O (log n)，快速幂的循环次数为 log n

2.空间复杂度：O (1)，仅用临时变量存储中间结果

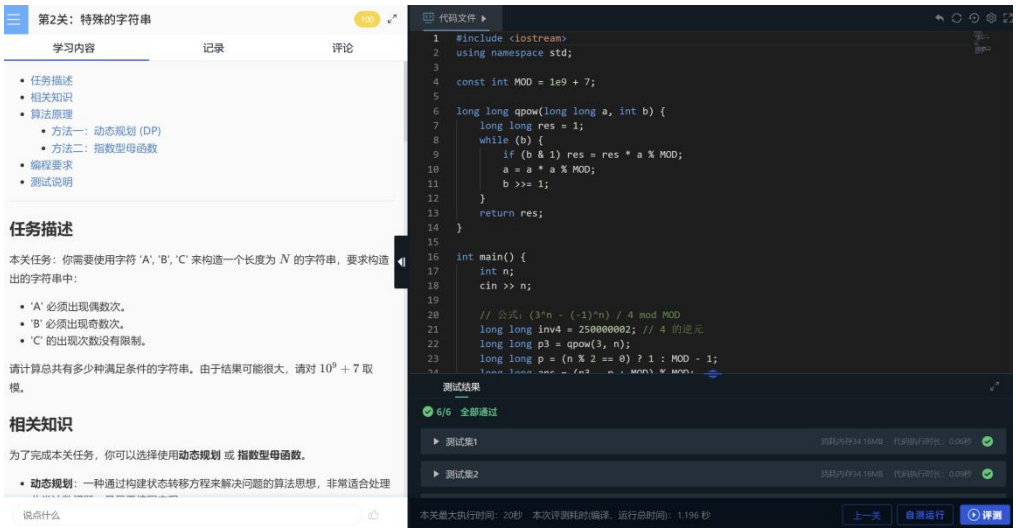
核心代码片段：

```
// 快速幂计算 a^b mod MOD
```

```
long long qpow(long long a, int b) {
    long long res = 1;
    while (b) {
        if (b & 1) res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

long long inv4 = 250000002; // 4 的逆元
long long p3 = qpow(3, n);
long long p = (n % 2 == 0) ? 1 : MOD - 1; //  处理(-1)^n
long long ans = (p3 - p + MOD) % MOD; //  防止负数
ans = ans * inv4 % MOD;
```

截图证明：



7　图论

7.1 实验目的

（1）掌握图的邻接矩阵表示方法，实现图、补图、导出子图的构造与输出。

（2）理解树的基本性质，掌握可树序列的判定方法。

（3）实现图的连通性判定（强连通、单向连通、弱连通）、可图序列判定（Havel-Hakimi 算法）。

（4）掌握邻接矩阵幂运算与通路计数的关系，实现指定长度通路的数量计算。

（5）学会判定欧拉图（欧拉回路 / 通路）、偶图（二分图），掌握 BFS、DFS、Floyd-Warshall 等经典图算法。

7.2 实验内容

7.2.1 树

（1）可树序列定理。给定一个度数序列，判断其是否可以构成一棵树。

7.2.2 图论的相关定义

（1）图的定义与邻接矩阵。给定一张图的点集和边集，求该图的邻接矩阵。

（2）子图与补图。给定一张图的点集和边集，输出其补图与指定的导出子图。

（3）图的同构判断。给定两张图的点集和边集，判断二者是否同构。

（4）图的连通性。给定有向图 G=<V,E>，判断其连通性。

7.2.3 图论的应用

（1）可图序列判定——Havel-Hakimi 算法。给定一个非负整数组，判断其是否为可图序列。

（2）求长度为 m 的通路个数。给定一张简单有向图及其邻接矩阵，求长度为 m 的通路（含回路）的总数。

7.2.4 特殊图

（1）欧拉图的判定。给定 n 个点 m 条边的有向图，判断其是否是欧拉图；如不是，再判断是否存在欧拉通路。

（2）偶图的判定。给定 n 个点 m 条边的无向图，判断其是否为偶图（二分图）。

7.3 实验过程

7.3.1 树

（1）可树序列定理

解题思路：根据树的基本性质判断度数序列是否可构成树。n 个节点的树有且仅有 n-1 条边，总度数等于 2*(n-1)；当 n>1 时，树中不能存在度为 0 的孤立节点；n=1 时是特殊情况，仅当度数为 0 时合法。

算法复杂度分析：

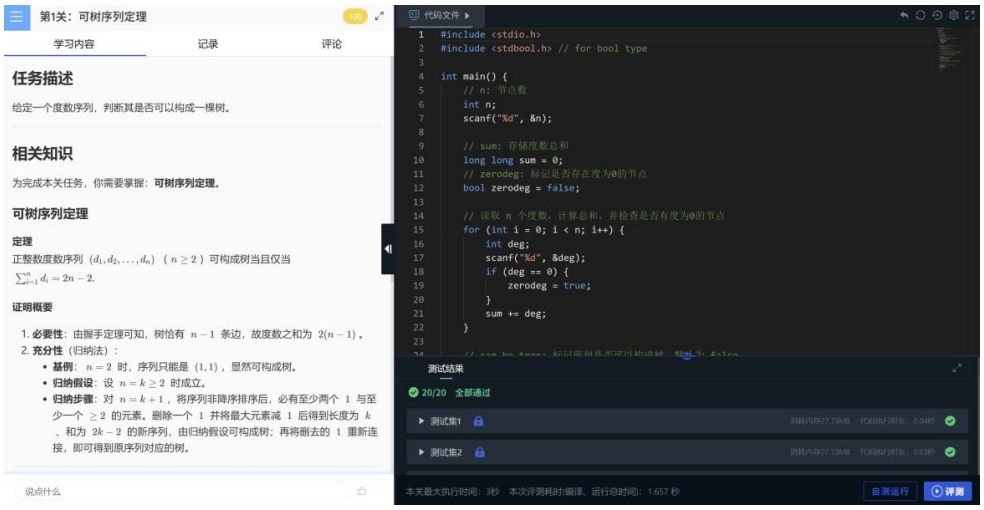
1. 时间复杂度：O (n)，仅遍历一次度数序列计算总和并检查孤立节点

2. 空间复杂度：O (1)，仅使用临时变量存储总和与标记

核心代码片段：

```
if (n == 1) {
    // 单个节点:度数必须为 0
    can_be_tree = (sum == 0);
} else {
    // 两个条件同时满足:度数和为 2(n-1)且无孤立节点
    can_be_tree = (sum == 2LL * (n - 1)) && (!zerodeg);
}
```

截图证明：



7.3.2 图论的相关定义

（1）图的定义与邻接矩阵

解题思路：初始化 n×n 的全 0 邻接矩阵，读取每条边的两个顶点，将顶点编号转换为 0 起始的数组下标，把矩阵对应位置设为 1，最终输出完整邻接矩阵。

算法复杂度分析：

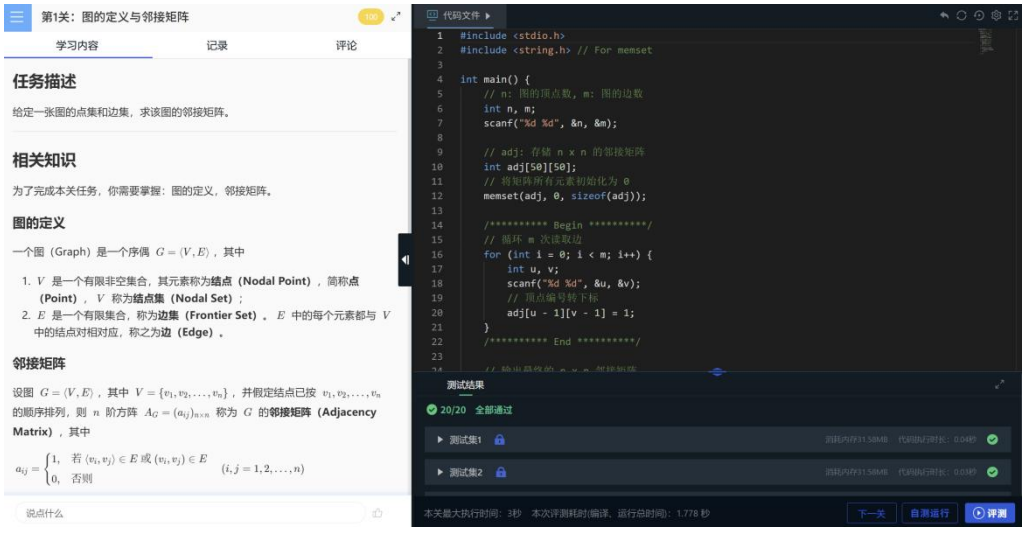
1. 时间复杂度：O（m），循环读取 m 条边并赋值

2. 空间复杂度：O（n^2），存储 n 行 n 列的邻接矩阵

核心代码片段：

```
// 初始化全 0 矩阵
memset(adj, 0, sizeof(adj));
// 读取每条边并设置对应位置为 1
for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u - 1][v - 1] = 1; // 1 起始转 0 起始下标
}
```

截图证明：



（2）子图与补图

解题思路：补图的邻接矩阵中，对角线元素恒为 0，其余位置为 1 减去原图对应位置的值；导出子图直接取原图邻接矩阵左上角 k×k 的部分，按格式输出即可。

算法复杂度分析：

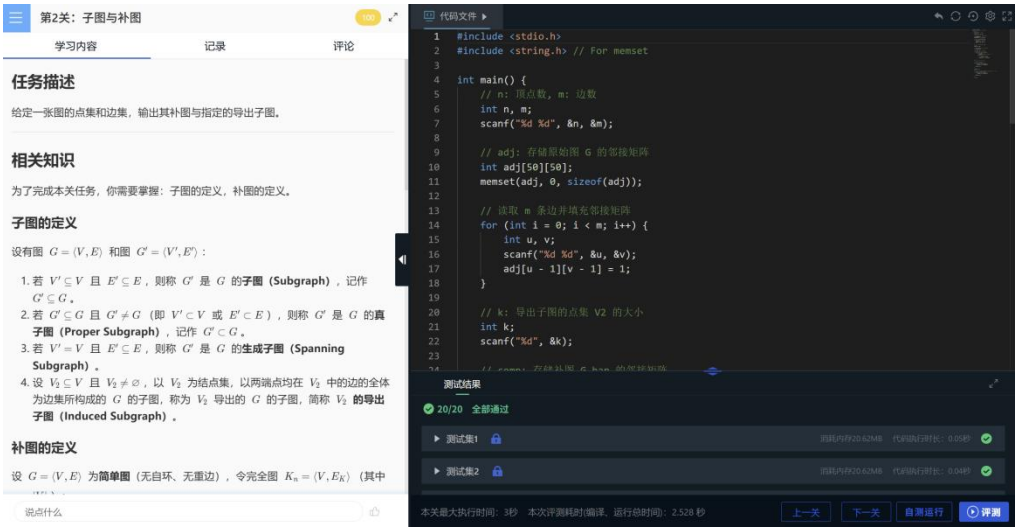
1. 时间复杂度：O（n^2），双重循环计算补图矩阵

2. 空间复杂度：O（n^2），存储原图与补图的邻接矩阵

核心代码片段：

```
// 计算补图邻接矩阵
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (i == j) {
            comp[i][j] = 0; // 自环永远为 0
        } else {
            comp[i][j] = 1 - adj[i][j]; // 原图无边则补图有边
        }
    }
}
```

截图证明：



（3）图的同构判断

解题思路：通过递归生成所有可能的顶点排列（映射关系），检查是否存在一种排列使得两个图的邻接关系完全一致。若存在则两图同构，否则不同构。

算法复杂度分析：

1. 时间复杂度：O（n! * n^2），生成 n! 种排列，每种排列检查 n^2 个邻接关系

2. 空间复杂度：O（n），存储排列数组与访问标记数组

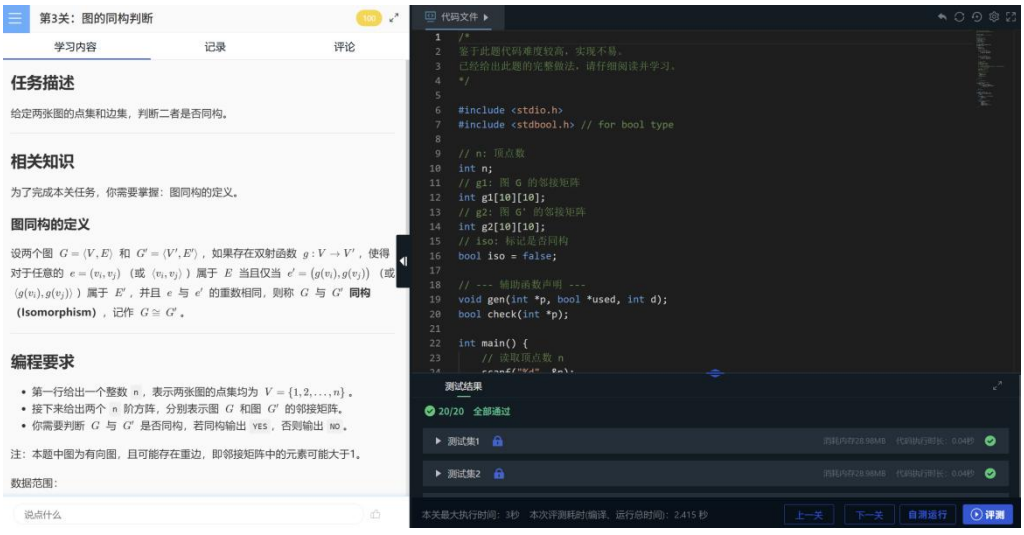
核心代码片段：

```
// 检查当前排列是否为同构映射
bool check(int *p) {
    for (int i = 0; i < n; i++) {
```

```
                for (int j = 0; j < n; j++) {
                    if (g1[i][j] != g2[p[i]][p[j]]) {
                        return false;
                    }
                }
            }
        return true;
    }

    // 递归生成所有排列并检查
    void gen(int *p, bool *used, int d) {
        if (iso) return; // 已找到同构映射，提前返回
        if (d == n) { // 生成完整排列
            if (check(p)) iso = true;
            return;
        }
        for (int i = 0; i < n; i++) {
            if (!used[i]) {
                p[d] = i;
                used[i] = true;
                gen(p, used, d + 1);
                used[i] = false; // 回溯
            }
        }
    }
```

截图证明：



（4）图的连通性

解题思路：先用 Floyd-Warshall 算法求有向图的传递闭包，判断任意两点是否可达。强连通要求任意两点互相可达；单向连通要求任意两点至少单向可达；

弱连通要求忽略方向后图连通；否则不连通。

算法复杂度分析：

1. 时间复杂度：O（n^3），Floyd-Warshall 算法三层循环

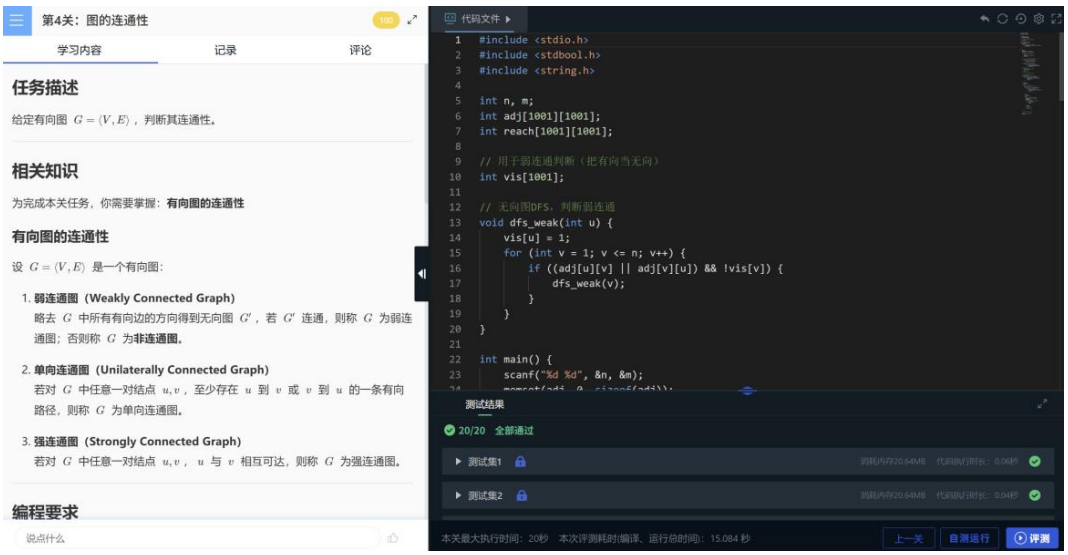
2. 空间复杂度：O（n^2），存储邻接矩阵与传递闭包矩阵

核心代码片段：

```
// Floyd-Warshall 求传递闭包
for (int k = 1; k <= n; k++)
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (reach[i][k] && reach[k][j])
                reach[i][j] = 1;

// 判断强连通
bool strong = true;
for (int i = 1; i <= n && strong; i++) {
    for (int j = 1; j <= n; j++) {
        if (!reach[i][j] || !reach[j][i]) {
            strong = false;
            break;
        }
    }
}
```

截图证明：



7.3.3 图论的应用

（1）可图序列判定 ——Havel-Hakimi 算法

解题思路：反复执行以下步骤：将序列降序排序，若全为 0 则可图；取第一个数 d，若 d≥序列长度或为负数则不可图；将后面 d 个数各减 1，删除第一个数，重复上述过程。

算法复杂度分析：

1.时间复杂度：O（n^2 log n），最多 n 次循环，每次排序耗时 O（n log n）

2.空间复杂度：O（n），存储度数序列

核心代码片段：

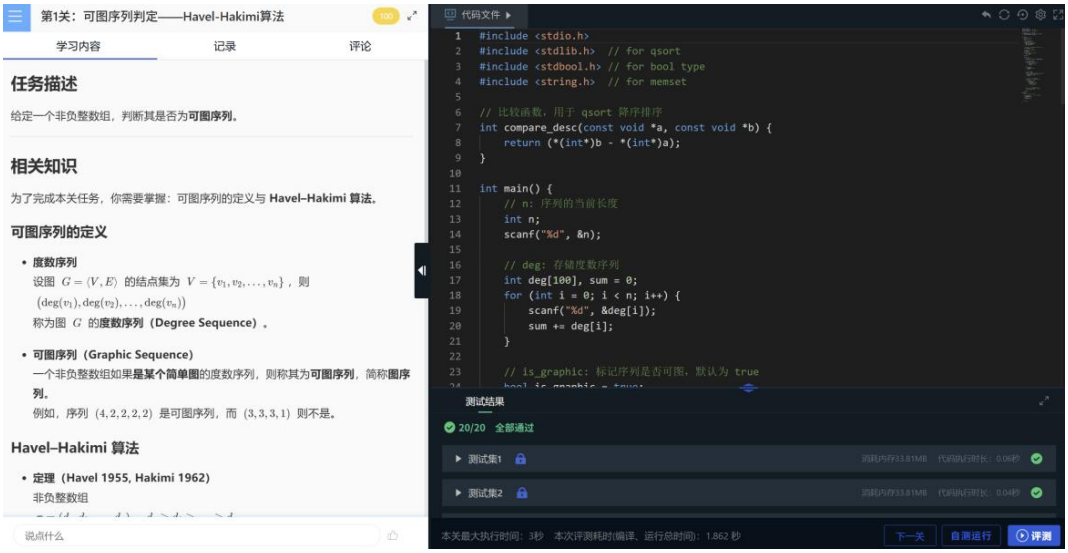
```
while (1) {
    qsort(d, len, sizeof(int), compare_desc); // 降序排序
    // 检查是否全 0
    bool all_zero = true;
    for (int i = 0; i < len; i++) {
        if (d[i] != 0) { all_zero = false; break; }
    }
    if (all_zero) { is_graphic = true; break; }

    int first = d[0];
    if (first < 0 || first >= len) { is_graphic = false; break; }

    // 后面 first 个数各减 1
    for (int i = 1; i <= first; i++) {
        d[i]--;
        if (d[i] < 0) { is_graphic = false; goto end_check; }
    }

    // 删除第一个元素
    for (int i = 0; i < len - 1; i++) d[i] = d[i + 1];
    len--;
}
```

截图证明：



（2）求长度为 m 的通路个数

解题思路：邻接矩阵的 m 次幂中，元素 A^m [i][j] 表示从 i 到 j 长度为 m 的通路个数。通过矩阵乘法计算邻接矩阵的 m 次幂，最后求和矩阵所有元素得到总通路数。

算法复杂度分析：

1. 时间复杂度：O（m * n³），进行 m 次矩阵乘法，每次乘法三层 n 阶循环

2. 空间复杂度：O（n²），存储邻接矩阵、结果矩阵与临时矩阵

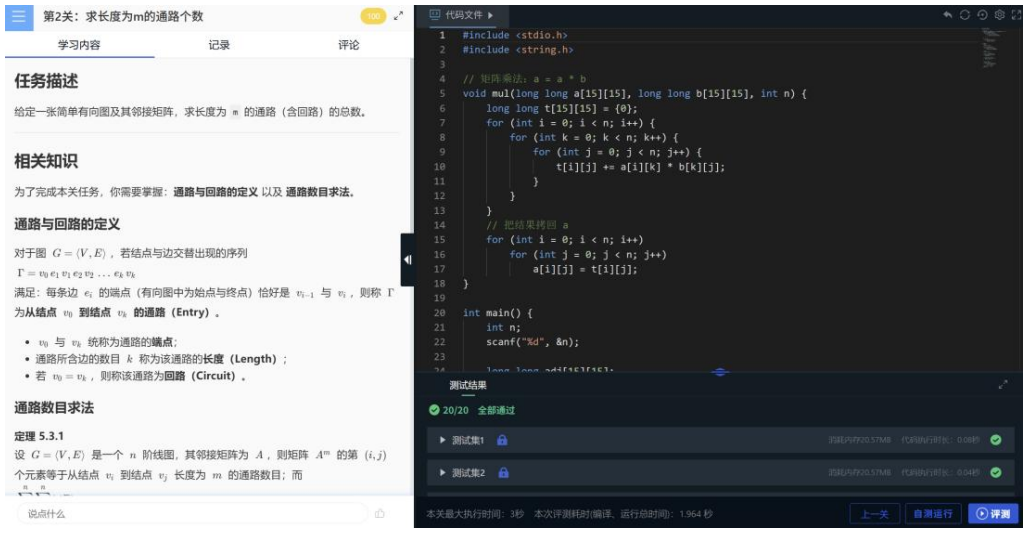
核心代码片段：

```
// 矩阵乘法:a = a * b
void mul(long long a[15][15], long long b[15][15], int n) {
    long long t[15][15] = {0};
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < n; k++) {
            for (int j = 0; j < n; j++) {
                t[i][j] += a[i][k] * b[k][j];
            }
        }
    }
    // 结果拷回 a
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            a[i][j] = t[i][j];
}

// 求邻接矩阵的 m 次幂
for (int i = 0; i < m; i++) {
```

```
        mul(res, adj, n);
    }
```

截图证明：



7.3.4 特殊图

（1）欧拉图的判定

解题思路：有向图是欧拉图（存在欧拉回路）当且仅当弱连通且所有顶点入度等于出度；存在欧拉通路当且仅当弱连通且恰好一个顶点出度比入度多 1（起点）、一个顶点入度比出度多 1（终点），其余顶点入度等于出度。

算法复杂度分析：

1. 时间复杂度：O（n + m），DFS 遍历图与统计度数

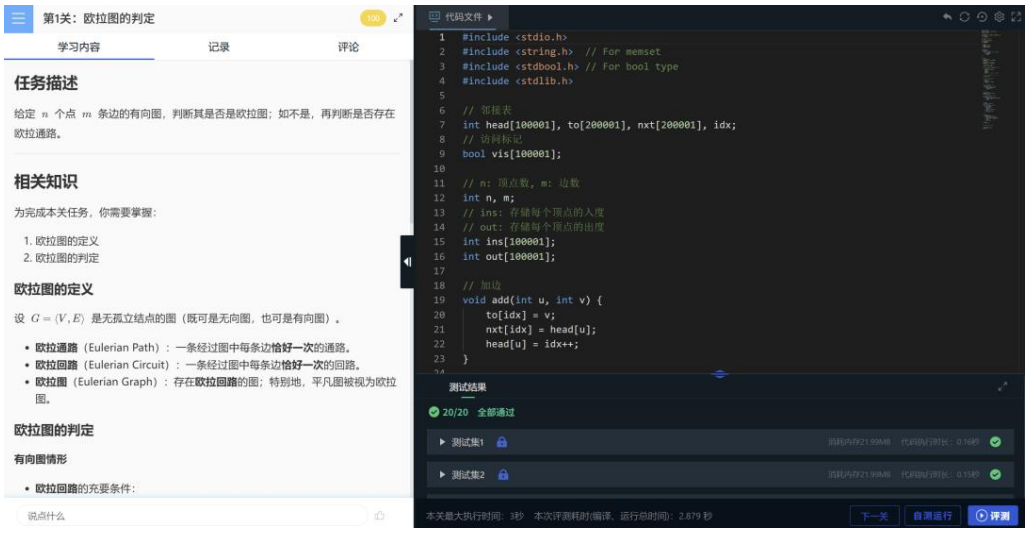
2. 空间复杂度：O（n + m），存储邻接表与访问标记

核心代码片段：

```
// 统计度数并检查条件
int start = 0, end = 0;
bool ok = true;
for (int v = 1; v <= n; v++) {
        int od = out[v];
        int id = ins[v];
        if (od == id + 1) {
                start++; // 起点计数
        } else if (id == od + 1) {
                end++;     // 终点计数
        } else if (od != id) {
                ok = false; // 度数不满足条件
        }
}
```

```
// 最终判断
if (connect && ok) {
        if (start == 0 && end == 0) {
                result = 2;     // 欧拉回路
        } else if (start == 1 && end == 1) {
                result = 1;     // 欧拉通路
        } else {
                result = 0;
        }
}
```

截图证明：



（2）偶图的判定

解题思路：采用 BFS 染色法，将图中顶点染成两种颜色，相邻顶点颜色必须不同。若能完成染色则是偶图（二分图），否则不是。需要处理非连通图，遍历所有未染色顶点。

算法复杂度分析：

1. 时间复杂度：O（n + m），BFS 遍历所有顶点与边

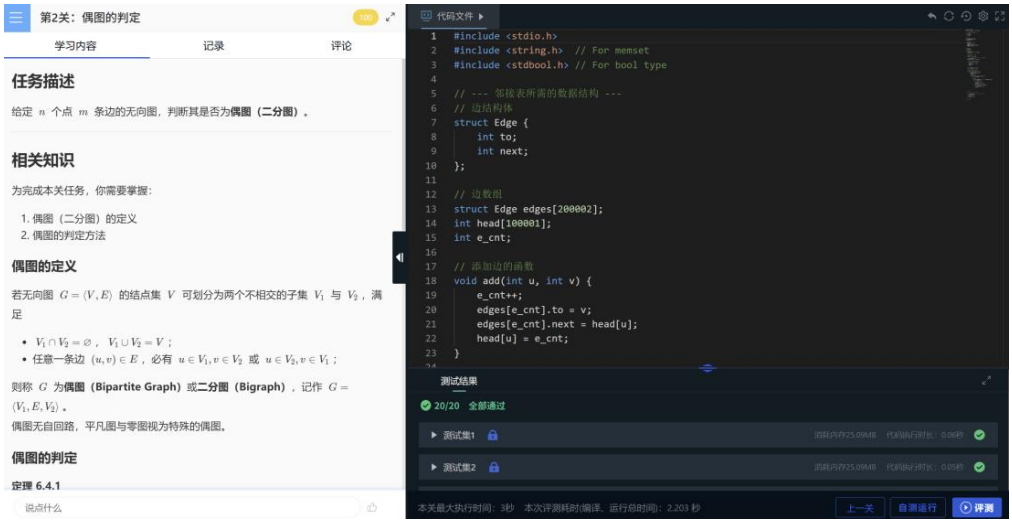
2. 空间复杂度：O（n + m），存储邻接表、颜色数组与队列

核心代码片段：

```
// BFS 染色
for (int i = 1; i <= n; ++i) {
        if (color[i] == 0) { // 未染色的连通分量
                int front = 0, rear = 0;
                q[rear++] = i;
                color[i] = 1;
```

```
            while (front < rear && is_bipartite) {
                int u = q[front++];
                // 遍历所有邻接点
                for (int j = head[u]; j != -1; j = edges[j].next) {
                    int v = edges[j].to;
                    if (color[v] == 0) {
                        color[v] = 3 - color[u]; // 染成相反颜色
                        q[rear++] = v;
                    } else if (color[v] == color[u]) {
                        is_bipartite = false; // 相邻同色，不是二分图
                        break;
                    }
                }
            }
        }
    }
```

截图证明：



8　数论（选做）

8.1 实验目的

（1）实现埃拉托斯特尼筛法，高效生成指定范围内的所有素数。

（2）掌握素因数分解、因数个数定理、欧拉函数的计算方法。

（3）应用欧拉定理处理超大指数模幂运算，拓展快速幂的适用场景。

（4）求解一般线性同余方程（不限制互质条件），全面掌握同余方程的解判定与求解方法。

（5）深化数论算法的理解与实现能力，为高级数论应用与密码学拓展奠定基础。

8.2 实验内容

8.2.1 数论

（1）埃拉托斯特尼筛法。本关任务：使用埃拉托斯特尼筛法生成小于等于给定上限的所有素数。

（2）计算因数个数。本关任务：输入两个正整数 N_1 和 N_2，计算它们的乘积 $P=N_1\times N_2$ 的正因子（约数）的个数。

（3）欧拉函数。本关任务：编写一个程序，求解给定正整数 n 的欧拉函数 $\phi(n)$ 的值。

（4）欧拉定理的应用。本关任务：应用欧拉定理与快速幂算法，高效地计算模幂 $a^b \pmod m$ 的值，其中 b 可能是一个位数极多的大整数。

（5）一般线性同余方程。本关任务：编写一个程序，求解形如 $ax\equiv b \pmod m$ 的线性同余方程。

说明：本关任务不保证 a 和 m 互质，你需要使用扩展欧几里得算法进行求解，并判断方程是否有解。

8.3 实验过程

8.3.1 数论

（1）埃拉托斯特尼筛法

解题思路：初始化布尔数组标记所有数为素数，0 和 1 标记为非素数；从 2 开始遍历到 sqrt (n)，若当前数是素数，则将其所有倍数（从平方开始）标记为非素数；最后收集所有标记为素数的数并输出。

算法复杂度分析：

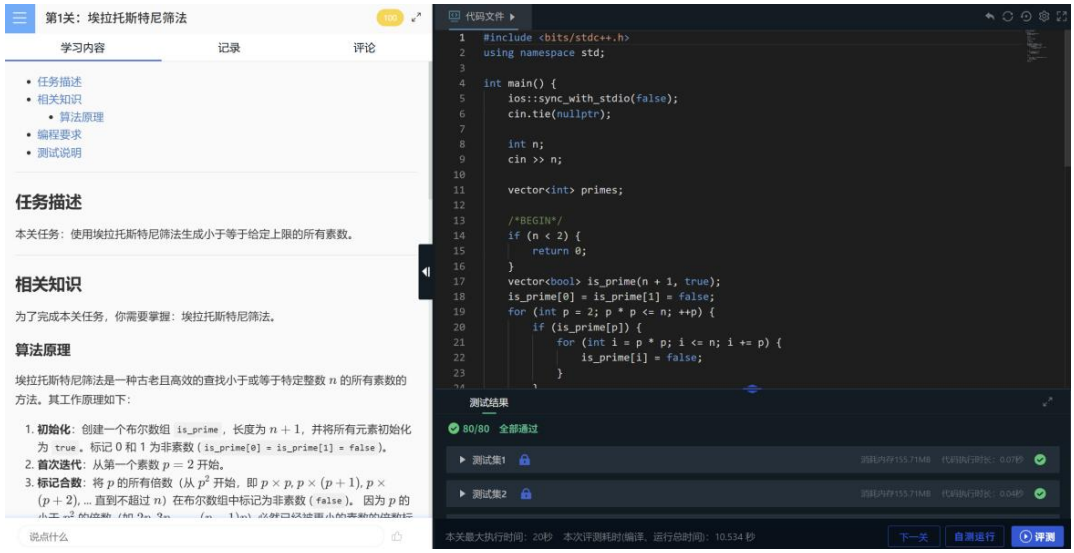
1. 时间复杂度：O (n log log n)，埃氏筛法的标准时间复杂度

2. 空间复杂度：O (n)，使用布尔数组存储素数标记

核心代码片段：

```
vector<bool> is_prime(n + 1, true);
is_prime[0] = is_prime[1] = false;
// 筛法核心：标记素数的倍数
for (int p = 2; p * p <= n; ++p) {
    if (is_prime[p]) { // p 是素数
        for (int i = p * p; i <= n; i += p) {
            is_prime[i] = false; // 标记 p 的倍数为非素数
        }
    }
}
// 收集所有素数
for (int i = 2; i <= n; ++i) {
    if (is_prime[i]) primes.push_back(i);
}
```

截图证明：



（2）计算因数个数

解题思路：根据因数个数定理，一个数的正因数个数等于其素因数分解中各指数加 1 的乘积。分别分解 N1 和 N2 的质因数，合并相同质因子的指数，最后计算所有指数加 1 的乘积得到结果。

算法复杂度分析：

1.时间复杂度：O（sqrt（n1）+ sqrt（n2）），试除法分解两个数的质因数

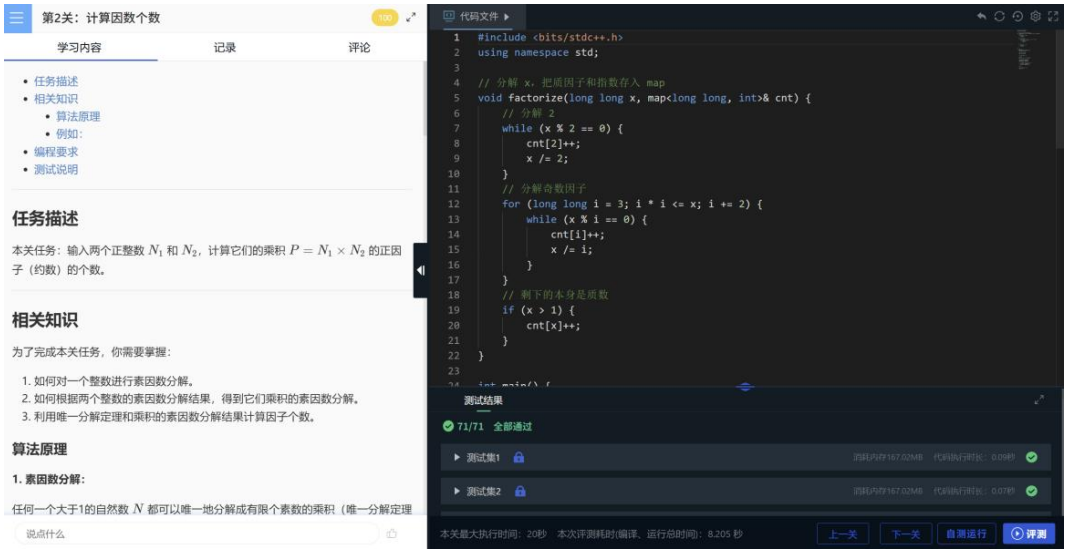
2.空间复杂度：O（因子个数），用 map 存储质因子与指数

核心代码片段：

```
// 分解 x 的质因数，存入 map
void factorize(long long x, map<long long, int>& cnt) {
    while (x % 2 == 0) { // 分解 2
        cnt[2]++;
        x /= 2;
    }
    for (long long i = 3; i * i <= x; i += 2) { // 分解奇数因子
        while (x % i == 0) {
            cnt[i]++;
            x /= i;
        }
    }
    if (x > 1) cnt[x]++; // 剩余的大质因子
}

// 合并两个数的质因数
factorize(n1, factors);
factorize(n2, factors);
// 计算因数总数：(e1+1)*(e2+1)*...
for (auto& p : factors) {
    ans *= (p.second + 1);
}
```

截图证明：



（3）欧拉函数

解题思路：欧拉函数 $\phi(n)$ 表示小于等于 n 且与 n 互质的正整数个数。公式为 $\phi(n)=n\times\prod(1-1/p)$，其中 p 是 n 的所有不同质因子。分解 n 的质因子，逐个应用公式计算，每个质因子只计算一次。

算法复杂度分析：

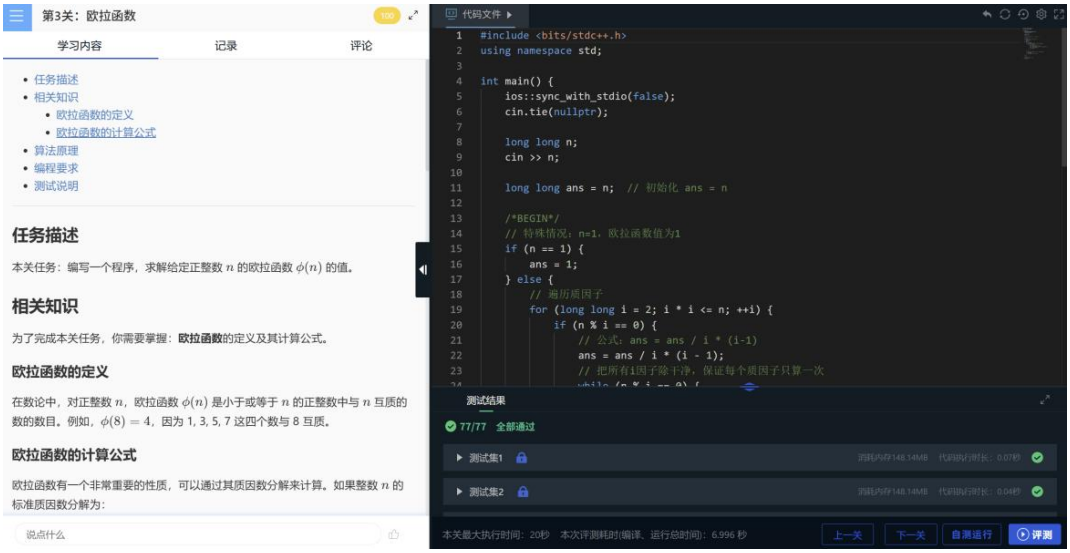
1. 时间复杂度：O（sqrt（n）），试除法分解质因数

2. 空间复杂度：O（1），仅使用临时变量

核心代码片段：

```
long long ans = n;
if (n == 1) {
    ans = 1; // 特殊情况：φ(1)=1
} else {
    for (long long i = 2; i * i <= n; ++i) {
        if (n % i == 0) { // i 是质因子
            ans = ans / i * (i - 1); // 应用公式
            while (n % i == 0) n /= i; // 除尽所有 i 因子
        }
    }
    if (n > 1) { // 剩余的大质因子
        ans = ans / n * (n - 1);
    }
}
```

截图证明：



（4）欧拉定理的应用

解题思路：根据欧拉定理，若 a 与 m 互质，则 a^b ≡ a^(b mod φ(m)) (mod m)。先计算 m 的欧拉函数 φ(m)，将大指数 b（字符串形式）对 φ(m) 取模，再用快速幂计算 a 的模幂结果。

算法复杂度分析：

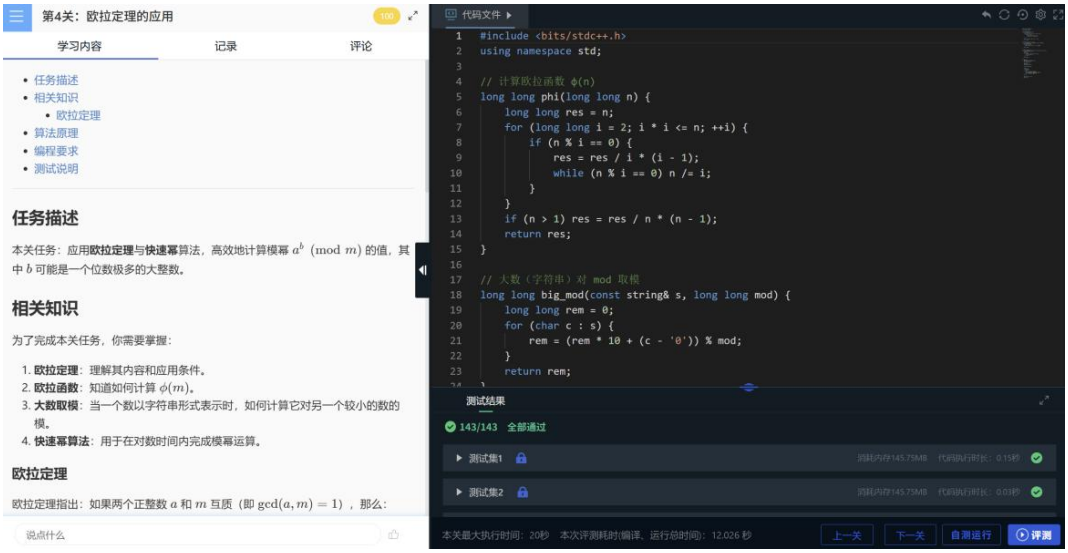
1. 时间复杂度：O (sqrt (m) + len (b) + log φ(m))，分别对应求欧拉函数、大指数取模、快速幂

2. 空间复杂度：O (1)，仅使用临时变量

核心代码片段：

```
if (m == 1) {
    ans = 0; // 任何数模 1 都是 0
} else {
    long long phi_m = phi(m); // 计算φ(m)
    long long b_mod = big_mod(b, phi_m); // 大指数 b 对φ(m)取模
    ans = mod_pow(a, b_mod, m); // 快速幂计算结果
}
```

截图证明：



（5）一般线性同余方程

解题思路：求解 ax ≡ b (mod m)，用扩展欧几里得算法求 gcd (a,m)。若 b 不能被 gcd 整除，则方程无解；否则求特解 x0 = x*(b/gcd)，解的间隔为 m/gcd，将特解规范化为最小正整数解。

算法复杂度分析：

1. 时间复杂度：O (log min (a,m))，扩展欧几里得算法的时间复杂度

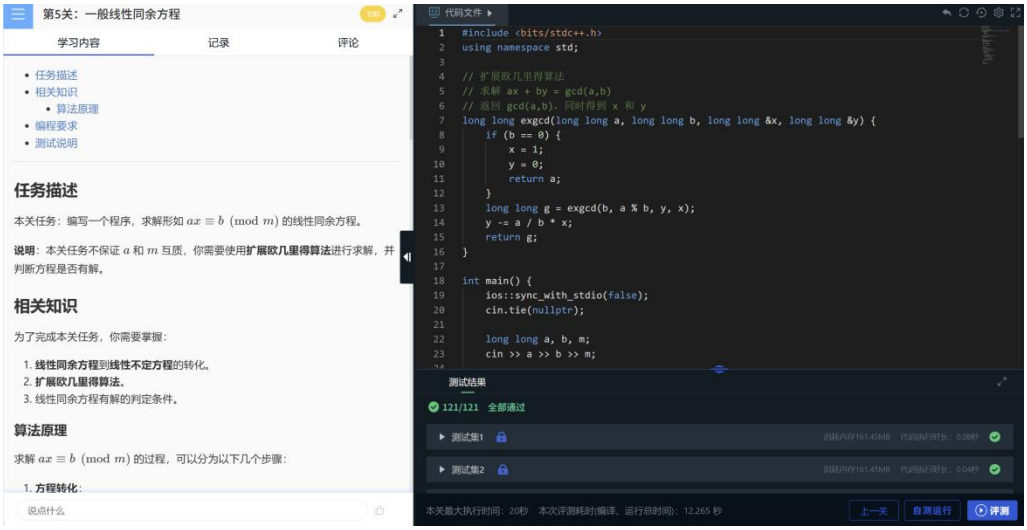
2. 空间复杂度：O (1)，仅使用临时变量

核心代码片段：

```
long long x, y;
long long g = exgcd(a, m, x, y); // 求 gcd(a,m)和系数 x,y

if (b % g != 0) {
    ans = -1; // 无解
} else {
    long long x0 = x * (b / g); // 求特解
    long long t = m / g; // 解的间隔
    ans = (x0 % t + t) % t; // 规范化为最小正解
}
```

截图证明：


第5关：一般线性同余方程
学习内容
记录
评论
任务描述
相关知识
算法原理
编程要求
测试说明
任务描述
本关任务：编写一个程序，求解形如 $ax \equiv b \pmod{m}$ 的线性同余方程。
说明：本关任务不保证 a 和 m 互质，你需要使用扩展欧几里得算法进行求解，并判断方程是否有解。
相关知识
为了完成本关任务，你需要掌握：
1. 线性同余方程到线性不定方程的转化。
2. 扩展欧几里得算法。
3. 线性同余方程有解的判定条件。
算法原理
求解 $ax \equiv b \pmod{m}$ 的过程，可以分为以下几个步骤：
1. 方程转化：
说点什么
代码文件
#include <bits/stdc++.h>
using namespace std;
// 扩展欧几里得算法
// 求解 ax + by = gcd(a,b)
// 返回 gcd(a,b)，同时得到 x 和 y
long long exgcd(long long a, long long b, long long &x, long long &y) {
if (b == 0) {
x = 1;
y = 0;
return a;
}
long long g = exgcd(b, a % b, y, x);
y -= a / b * x;
return g;
}
int main() {
ios::sync_with_stdio(false);
cin.tie(nullptr);
long long a, b, m;
cin >> a >> b >> m;
测试结果
121/121 全部通过
测试集1
测试集2
本关最大执行时间：20秒 本次评测耗时(编译、运行总时间)：12.265 秒
上一关
自测运行
评测

9　总结与课程建议

9.1 课程总结

本次离散数学编程实践课程，系统覆盖了数理逻辑、集合及其运算、关系表示及其运算、函数、初等数论、组合数学、图论及拓展数论八大核心模块，将抽象的离散数学理论与计算机编程实践深度结合，实现了从数学概念到可执行代码的转化，在知识掌握、能力提升和思维培养三个维度均收获显著。

在知识体系构建上，我完整掌握了离散数学各分支的核心原理与应用方法。数理逻辑部分，学会了用符号化语言表示命题与谓词，掌握了推理有效性验证、真值表生成、等价演算及归结原理等基础逻辑工具；集合与关系部分，实现了集合的幂集生成、容斥原理计算，以及关系矩阵、关系幂运算、传递闭包（Warshall 算法）和特殊关系（等价关系、偏序关系）的判定与应用；函数部分，完成了函数定义验证、单射 / 满射 / 双射判断及函数复合运算；数论部分，深入理解了模运算、素数判定、扩展欧几里得算法、同余方程求解及 RSA 密码学原理；组合数学部分，掌握了计数原理、鸽巢原理、排列组合及生成函数的应用；图论部分，实现了图的邻接矩阵表示、子图与补图计算、连通性判定、可图序列验证及欧拉图、偶图等特殊图的识别。

在编程实践能力上，我熟练运用 C/C++ 语言实现了各类离散数学算法，解决了多个典型问题。针对大数运算场景，掌握了逐位取模、字符串处理等技巧；针对图论问题，学会了邻接矩阵与邻接表两种存储方式，以及 DFS、BFS、Floyd-Warshall 等经典图算法；针对数论与组合问题，熟练运用快速幂、筛法、动态规划等算法思想优化计算效率。同时，养成了规范的代码编写习惯，能够清晰标注核心逻辑注释，保证代码的可读性与可维护性。

在思维培养上，离散数学的严谨性与抽象性极大锻炼了我的逻辑推理与数学建模能力。通过将实际问题抽象为数学模型（如将密码学问题转化为同余方程、将计数问题转化为生成函数卷积），学会了用数学方法分析和解决计算机领域的实际问题。例如，利用鸽巢原理巧妙解决子集和与整中点问题，利用动态规划处理带约束的计数问题，这些实践让我深刻体会到离散数学作为计算机科学基础的

核心价值。

9.2 问题及建议

9.2.1 存在的问题

（1）部分实验算法效率不足：部分实验采用了暴力枚举的方法，如图同构判断的全排列搜索，仅能处理 n≤10 的小规模数据，无法应对实际应用中的大规模图；唯一分解定理的试除法在处理大整数时效率较低，缺乏更高效的算法实现。

（2）理论与实践衔接不够紧密：部分章节的理论推导与代码实现存在断层，例如生成函数的数学推导过程较为抽象，但实验中直接给出了最终公式，未能充分体现从理论到代码的转化逻辑；欧拉定理的应用实验中，对定理适用条件的讲解不够深入，容易导致对算法原理的理解偏差。

（3）实验测试用例覆盖不足：部分实验的测试用例较为单一，主要覆盖常规场景，对边界条件和异常情况的测试不够充分。例如，线性同余方程实验中，对无解情况的测试用例较少；图的连通性实验中，对非连通图、孤立节点等特殊情况的验证不够全面。

（4）应用场景拓展不足：现有实验多聚焦于基础算法实现，缺乏离散数学在人工智能、数据结构、计算机网络等领域的实际应用案例。例如，图论部分未涉及最短路径、最小生成树等经典应用问题，数理逻辑部分未结合专家系统、自动推理等实际场景。

9.2.2 改进建议

（1）优化实验算法设计：补充更高效的算法实现，如图同构判断可引入基于度数序列和邻接结构的剪枝算法，大整数分解可引入 Pollard Rho 算法；同时增加算法对比环节，让学生直观感受不同算法的时间复杂度差异。

（2）强化理论与实践结合：在实验指导中增加理论推导到代码实现的分步讲解，例如生成函数实验中，详细展示从指数型母函数到最终公式的推导过程；在关键算法处标注对应的数学原理，帮助学生理解代码背后的数学逻辑。

（3）丰富实验测试用例：增加边界条件和异常情况的测试用例，如空集、孤立节点、无解方程、大数值输入等；同时提供本地调试工具，让学生能够自行

测试代码的鲁棒性，减少因格式错误或边界问题导致的评测失败。

（4）拓展应用场景实验：增加离散数学在实际领域的应用实验，例如用图论实现最短路径算法解决导航问题，用数理逻辑实现简单的规则推理系统，用组合数学解决背包问题等；通过真实场景的应用，让学生更深刻地理解离散数学的实用价值。

（5）增加开放性实验环节：设置部分开放性题目，让学生自主选择问题并设计解决方案，例如 “基于 RSA 算法实现简单的加密通信程序”“用图论分析校园网络拓扑结构” 等，培养学生的自主创新能力和工程实践能力。